Employing Metaprogramming: A Macro-Driven Strategy to Overcome Compiler-Imposed Limitations on Specialization

Davide Cerato
Id. Number: 984481

BSc in Computer Science

Advisor: Prof. Walter Cazzola
Co-Advisor: M.Sc. Federico Bruzzone





UNIVERSITÀ DEGLI STUDI DI MILANO
Computer Science Department
ADAPT-Lab

Academic Year 2024–2025

COMPUTER SCIENCE DEPARTMENT • ADAPT-LAB

Contents

1 Introduction **1**

2 Running Example **3**

3 Background **7**
3.1 The Rust Programming Language. 7
3.2 Dispatch Techniques 9
3.3 Metaprogramming 11

4 A deep dive into MonoSpec **13**
4.1 Introduction 13
4.2 Conditions Definition 14
4.3 Method Wrapping 18
4.4 Build Script 20
4.5 Lifetimes 21
4.6 Limitations 25

5 Evaluation **27**
5.1 Introduction 27
5.2 Results 33
5.3 Considerations 35
5.4 Data 37

6 Threats to Validity **43**
6.1 Construct Validity 43
6.2 Internal Validity 43
6.3 External Validity 44
6.4 Conclusion Validity 44

7 Related Work **45**

8 Future Work **49**

9 Conclusions **51**

1 Introduction

Premise A core challenge in high-performance systems programming is achieving zero-cost abstraction [68, 70] through generic programming [56, 7] and polymorphism [13, 76] while allowing type-specific performance optimizations. This concept is known as *specialization* and it's a common aspiration in advanced type systems. This challenge is particularly acute in Rust, a modern systems programming language designed for safety, concurrency and performance. Its trait system enable polymorphism and generic programming without incurring runtime overhead, as the compiler typically performs monomorphization [34], ensuring high performance across the board. However, for maximum efficiency, there are instances where a generic function could benefit significantly from a custom, highly optimized implementation when dealing with specific types / conditions. The ability to provide such a conditional, type-specific implementation for a trait method is known as Trait Specialization.

State of the Art Many programming languages have explored various mechanisms to achieve specialization, with varying degrees of success and complexity. Without native support for specialization, in many cases the common practice is to rely on various complex workarounds which often result in cumbersome code, poor compiler diagnostics or insufficient optimization compared to true specialization. In 2015, the Rust community introduced an experimental feature called *trait specialization* [60], which soon received widespread interest, but up to now is still confined to the *Nightly* channel, after its stabilization attempts stalled due to various design and implementation challenges.

Limitations A general obstacle for adopting powerful, cutting-edge language features is the tension between innovation and stability, often manifested in experimental versus production-ready compiler versions. This is true for specialization as well, forcing library authors and application developers to make an undesirable trade-off: either forgo the performance and elegance improvements offered by specialization to maintain stability or commit to volatile and constantly changing unstable versions. For example, the primary and most critical limitation facing the Rust ecosystem is indeed the Nightly-Stable divide regarding specialization (the difference between these channels is explained in Section 3.1). While the feature offers significant potential for optimization and cleaner trait designs, its unstable status prevents its adoption in commercial, production-ready or long-term supported codebases.

Problem Statement While the concept of specialization is a crucial tool for optimizing generic code in any performance-oriented language, its practical application is often gated by compiler maturity and stability concerns. The lack or instability of specialization in programming languages creates a gap between what is theoretically possible for optimization and what is practically available to the vast majority of the community, putting a performance ceiling on stable code. Specifically for Rust, given its performance focus and the significant utility of trait specialization, the central problem is the lack of a robust, idiomatic and performance-preserving mechanism for implementing trait specialization in stable code.

Proposal To address the general limitations of specialization features in current stable ecosystems, this thesis proposes an innovative approach to the matter by moving the monomorphization and specialization logics to an earlier stage in the compilation process. To prove the effectiveness of this approach, we introduce a metaprogramming-based technique to simulate the behavior of compiler-implemented specialization. By utilizing the metaprogramming capabilities of the language [52], we can analyze the input and generate the necessary boilerplate and type-checking logic at compile time, effectively allowing a user to define specialized implementations in stable code, avoiding runtime overhead entirely and circumventing the restrictions imposed by unstable compiler features. This approach can be applied to most languages with metaprogramming support and even directly into the compiler frontend of modern languages, as the compiler can simply replicate our approach internally.

Contributions This work provides the following key contributions. A Novel Stable Specialization Technique: the design and implementation of a new metaprogramming-based approach for achieving specialization. Implementation of Reusable Tools: the creation of a Rust procedural macro library that provides an alternative to nightly specialization and an analysis tool that evaluates real-world public Rust codebases to identify common patterns where this stable specialization mechanism could yield substantial benefits.

Structure The remainder of this thesis is structured as follows. Chapter 2 (Running Example) illustrates the tool in action through a concrete example. Chapter 3 (Background) provides foundational knowledge on Rust, dispatch techniques and metaprogramming. Chapter 4 (A deep dive into MonoSpec) details the design and architecture of the specialization macro tool. Chapter 5 (Evaluation) presents the evaluation methodology and results, including benchmarks and ecosystem analysis. Chapter 6 (Threats to Validity) discusses the threats to validity of the evaluation. Chapter 7 (Related Work) reviews existing solutions for specialization in Rust and related languages. Chapter 8 (Future Work) outlines potential extensions and further research avenues. Finally, Chapter 9 (Conclusions) summarizes the findings and concludes the thesis.

2
Running Example

The simplest form of specialization using the tool is the following. We'll show the example in Rust, but the approach is general and can be applied to other languages with similar features.

Let's consider a trait `MyTrait` with a method `my_method`, and we want to have a generic implementation as well as provide a specialized implementation for the type `i32`. In standard Rust this would look something like Listing 1, but unfortunately this code would not compile due to the lack of stable specialization features. More specifically, the Rust compiler would produce the error message shown in Listing 2, indicating that the specialized implementation conflicts with the generic one.

```rust
trait MyTrait {
    fn my_method(&self);
}
impl<T> MyTrait for T {
    fn my_method(&self) {
        println!("Generic implementation");
    }
}
impl MyTrait for i32 {
    fn my_method(&self) {
        println!("Specialized for i32");
    }
}
fn main() {
    10.my_method();   // should print "Specialized for i32"
    10.5.my_method(); // should print "Generic implementation"
}
```

Listing 1. *Example of how specialization should work.*

```
error[E0119]: conflicting implementations of trait MyTrait for type i32
 --> src/main.rs:9:1
  |
4 | impl<T> MyTrait for T {
  | --------------------- first implementation here
...
9 | impl MyTrait for i32 {
  | ^^^^^^^^^^^^^^^^^^^^ conflicting implementation for i32
```

Listing 2. *Error message produced by the Rust compiler for the code in Listing 1.*

To solve this problem we can use MonoSpec, which allows us to define specialized implementations of trait methods for specific types without running into the conflicts that Rust's type system would normally prevent.

Using the tool, the default implementation does not need any change compared to standard Rust code. The specialized implementation, instead, is annotated with the attribute macro `#[when(...)]`, that lets us specify the conditions under which that implementation should be used. In this case, we want to specialize the implementation of `MyTrait` for the type `i32`, so we write `#[when(T = i32)]`, indicating that this implementation should be used whenever the generic parameter `T` corresponds to `i32`. This will create a specialized version of the `trait` and of the `impl` for `i32`, replacing in them all occurrences of the generic parameter `T` with our specialized type, through a process we'll be calling *monomorphic specialization*.

After doing this, we also need to instruct the method calls to use the correct version of the method. This is done by using the `spec!` macro, which takes as first argument the method call you'd like to specialize and as second argument the type to specialize for. As better explained in Chapter 4, this is required because in the macro expansion phase, the Rust compiler doesn't give us enough information to automatically determine the types of the variables involved in the method call, so we need to provide that information ourselves. This extra piece of information allows the macro to rewrite the method call to use the appropriate version of the method, but could be omitted in languages with more powerful macro systems or reflection capabilities.

The complete code is shown in Listing 3, where you can see the macros in action. As you can probably understand, this approach preserves the soundness of the type system, since after the monomorphization process (see Listing 4) we have two completely separate versions of the trait and its implementation, that the compiler can easily distinguish and type check independently.

```
1  trait MyTrait {
2      fn my_method(&self);
3  }
4  impl<T> MyTrait for T {
5      fn my_method(&self) {
6          println!("Generic implementation");
7      }
8  }
9  #[when(T = i32)]
10 impl<T> MyTrait for T {
11     fn my_method(&self) {
12         println!("Specialized for i32");
13     }
14 }
15 fn main() {
16     spec! { 10.my_method(); i32 };   // prints "Specialized for i32"
17     spec! { 10.5.my_method(); f64 }; // prints "Generic implementation"
18 }
```

Listing 3. *Example of the tool usage.*

```rust
trait MyTrait {
    fn my_method(&self);
}
impl<T> MyTrait for T {
    fn my_method(&self) {
        println!("Generic implementation");
    }
}
trait MyTrait_i32 {
    fn my_method(&self);
}
impl MyTrait_i32 for i32 {
    fn my_method(&self) {
        println!("Specialized for i32");
    }
}
fn main() {
    <i32 as MyTrait_i32>::my_method(&10); // prints "Specialized for i32"
    <f64 as MyTrait>::my_method(&10.5);   // prints "Generic implementation"
}
```

Listing 4. *Example of the code after our macros' expansion.*

In addition to using the two macros, we need to instruct the compiler to allow us to define and use the macros correctly independently from the order of their definition. In Rust this is done with *build scripts*, as shown in Listing 5 and better explained later in Section 4.4. This step is also useful to collect all the traits and default implementations present in the codebase, so that the tool can generate the specialized versions as needed.

```rust
fn main() {
    mono_spec::handle_order();
}
```

Listing 5. *Example of the minimal content of the* `build.rs` *file.*

With this simple example, we have shown how to use MonoSpec to define a specialized implementation of a trait method for a specific type. In the next chapters, we will explore more complex scenarios, including multiple specializations, constraints on generic parameters and how the tool handles these cases internally.

3 Background

3.1 The Rust Programming Language.

Rust is a system programming language that focuses on safety, speed and concurrency. It ensures memory safety without garbage collection, meaning pure Rust programs are free from null pointer dereferences and data races.

Ownership and Borrowing Rust's ownership system, integrated into its type system, is inspired by *linear logic* [32, 33] and *linear types* [76, 58], enforcing that each piece of memory (a value) has a single *owner* (the variable binding) at any time [20, 11]. When the owner goes out of scope, the memory is automatically deallocated, enabling user-defined destructors and supporting the *resource acquisition is initialization* (RAII) pattern [69]. Ownership can be transferred (*moved*) or temporarily shared (*borrowed*) through references. Rust supports two types of borrows: multiple *immutable* borrows or a single *mutable* borrow, but never both at the same time. These constraints, enforced by the compiler through *borrow checking*, guarantee memory safety and prevent dangling pointers by ensuring that reference lifetimes never outlive their owners. To support low-level operations, Rust provides `unsafe` blocks, where the compiler's safety guarantees are suspended and the burden of avoiding undefined behavior falls on the programmer. Outside of these blocks, Rust enforces strict safety, making undefined behavior impossible in safe code.

Structs, Traits and Impls Structs (`struct`) and enums (`enum`) are the primary composite data types in Rust, enabling the creation of complex data structures with named fields and variant types, respectively.

```
struct MyStruct {
    field1: i32,
    field2: String,
}
```

Listing 6. *Example of a Rust struct.*

```
enum MyEnum {
    Variant1,
    Variant2(i32),
}
```

Listing 7. *Example of a Rust enum.*

Rust's trait system is a central mechanism for expressing polymorphism and shared behavior across types [46, 53]. Traits (`trait`) define a set of method signatures that types can implement, enabling abstraction over operations without committing to a concrete type. Implementation blocks (`impl`) associate specific types with traits, providing

concrete method definitions that satisfy the trait's contract. Rust also supports *inherent implementations*, where methods are defined directly on a type without requiring a trait, allowing type-specific functionality and constructor-like associated functions.

```
trait MyTrait { fn do_something(&self) }
impl MyTrait for i32 {
    fn do_something(&self) {}
}
```

Listing 8. *Example of a Rust trait impl.*

```
struct MyStruct;
impl MyStruct {
    fn do_something(&self) {}
}
```

Listing 9. *Example of a Rust inherent impl.*

By combining *traits*, *trait bounds* and *impl blocks*, Rust achieves ad hoc polymorphism [13, 76] and generic programming [56, 7] while preserving static guarantees and enabling the compiler to resolve method calls efficiently at compile time. Trait-based design is tightly integrated with Rust's ownership and borrowing model, ensuring that polymorphic abstractions do not compromise memory safety.

Software Variability Rust supports software variability through its attribute system, particularly the `cfg` attribute, which allows conditional compilation based on specified configuration predicates A feature in Rust is defined by all *terms* annotated with the same *config name* in a `cfg` *outer attribute*. The inclusion of a feature in a product depends on the evaluation of a *configuration predicate* at compile time, e.g. `#[cfg(any(unix, windows))]`. *Cargo*, Rust's package manager, lets developers declare features using *config names* in the `Cargo.toml` file, combining them with logical operators into *configuration predicates*. Feature dependencies, i.e., *cross-tree constraints* of *feature models* [42, 65, 43], can also be specified in the same file.

Release Channels Rust is distributed through three release channels: *stable*, *beta* and *nightly*. These channels differ in maturity guarantees and access to experimental language features [63]. The *stable* channel prioritizes reliability and backwards compatibility: only fully tested features are enabled, ensuring that libraries and applications compiled on stable continue to work across future releases. The *beta* channel serves as a staging ground for upcoming stable releases, allowing developers to test new features before they reach stable. In contrast, the *nightly* channel is updated daily and exposes experimental compiler internals, unstable language constructs and opt-in features (guarded by `#![feature(...)]` attributes). These features include advanced compiler hooks, low-level intrinsics and incomplete language extensions such as *specialization* and are available exclusively on nightly because their behavior is still under active design and soundness evaluation. Although nightly enables powerful experimentation, it provides no stability guarantees: features may change, be redesigned or removed without deprecation. For this reason, production-grade Rust codebases overwhelmingly target the stable channel, limiting developers from relying on certain expressive but unstable mechanisms.

3.2 Dispatch Techniques

Modern programming languages employ a variety of dispatch strategies at run-time or compile-time to determine which function or method is executed. These strategies fundamentally influence a language's expressiveness, performance guarantees and capacity for static reasoning [13, 55]. Rust integrates several dispatch paradigms to reconcile expressive abstraction with predictable performance, adhering to the principle of *zero-cost abstraction* [68, 70]. Most decisions are resolved statically, enabling the compiler to generate optimized machine code without runtime overhead [2, 25]. At the same time, Rust offers controlled forms of dynamic dispatch where late binding is essential, such as trait objects [53]. Beyond these core mechanisms, techniques like inlining heuristics [15], function cloning [26] and trait-based overloading [71] enable the compiler to further optimize code based on contextual information [17, 78]. Together, Rust's dispatch mechanisms provide a flexible yet robust model that preserves safety and performance.

Monomorphization Monomorphization [34] is the process by which the compiler generates concrete, type-specific versions of generic functions, methods and data structures. When a generic function is instantiated with a particular type parameter, the compiler produces a specialized version of the function in the final binary. This approach mirrors C++ template instantiation [75], enabling aggressive optimizations such as inlining, constant propagation and dead-code elimination [2, 78, 25].

```cpp
template <typename T>
T add(T a, T b) {
    return a + b;
}
int main() {
    // 1. Instantiation with int
    int x = add<int>(5, 3);
    // 2. Instantiation with double
    double y = add<double>(5.5, 3.2);
    return 0;
}
```

```cpp
int add_int(int a, int b) {
    return a + b;
}
double add_double(double a, double b) {
    return a + b;
}
int main() {
    int x = add_int(5, 3);
    double y = add_double(5.5, 3.2);
    return 0;
}
```

Listing 10. *Monomorphization process in C++: on the left, a generic function* `add<T>` *is defined and instantiated with two different types (*`int` *and* `double`*); on the right, the compiler generates two specialized versions of the function (*`add_int` *and* `add_double`*) and uses them accordingly.*

Because all type information is known at compile time, monomorphized functions avoid virtual dispatch overhead and allow the compiler to reason precisely about memory layout [51]. The primary trade-off is *code bloat*: large applications with many generic instantiations may experience increased binary size. Nevertheless, Rust's monomorphization strategy is central to its performance model, making generic abstractions effectively as fast as handwritten, type-specific code.

Overloading Overloading allows multiple functions or methods to share the same name but differ in parameter types or counts. Rust does not support traditional function overloading by parameter types as seen in languages like C++ or Java. Instead, it employs a unified mechanism based on *trait resolution* and *associated functions* [46]. Overloading emerges indirectly: different types may provide different implementations of the same trait methods and users may introduce trait bounds to guide method selection. This form of ad hoc polymorphism resembles *type classes* in Haskell [77, 35], where method resolution is determined by the combination of the type and the trait. Rust enforces coherence to guarantee that the compiler can unambiguously select an implementation at compile time [19, 30], preventing conflicts caused by independently defined overlapping instances. In addition, traits can define associated functions that behave similarly to static methods, allowing for API designs that mimic overloading while remaining type-safe and deterministic. This approach ensures predictable dispatch behavior, maintains clarity in method resolution and avoids the complexities of multi-parameter or ambiguous overload sets common in other languages [40].

Specialization Specialization is an advanced dispatch mechanism that allows more specific trait implementations to refine or override default behavior. In Rust, specialization aims to resolve long-standing expressiveness gaps in the trait system by enabling prioritized selection among overlapping implementations. This is particularly useful when a general-purpose implementation exists (often expressed as a *default* method in a trait) but certain types require optimized or semantically distinct behavior [60]. The feature builds on Rust's *coherence* and *orphan rules*[62], ensuring that overlapping implementations maintain type-safety and global consistency [19]. Although as of today specialization remains unstable and experimental (through the `#![feature(specialization)]` or `#![feature(min_specialization)]` flags [60, 61]), it provides a foundation for advanced library design patterns, such as optimized container types, numerical traits and fallback-based APIs. The mechanism is conceptually related to multi-methods and ad hoc polymorphism in other languages [16, 14], but Rust restricts its form to preserve determinism and avoid ambiguity in resolution that have yet to be fully addressed.

```cpp
template <typename T>
void myMethod(T value) {
    cout << "Generic implementation" << endl;
}

template <>
void myMethod<int>(int value) {
    cout << "Specialized for int" << endl;
}

int main() {
    myMethod(10);       // prints "Specialized for int"
    myMethod(10.5);     // prints "Generic implementation"
    return 0;
}
```

Listing 11. *Specialization in C++. The code is the equivalent of that in Listing 3.*

```rust
#![feature(specialization)]
trait MyTrait {
    fn my_method(&self);
}
impl<T> MyTrait for T {
    default fn my_method(&self) {
        println!("Generic implementation");
    }
}
impl MyTrait for i32 {
    fn my_method(&self) {
        println!("Specialized for i32");
    }
}
fn main() {
    10.my_method();     // prints "Specialized for i32"
    10.5.my_method();   // prints "Generic implementation"
}
```

Listing 12. *Specialization in Rust (unstable). The code is the equivalent of that in Listing 11.*

3.3 Metaprogramming

Metaprogramming refers to the practice of writing code that generates or manipulates other code, enabling abstraction, automation and domain-specific optimizations. Historically rooted in Lisp's symbolic macro systems [54] and later expanded through compile-time techniques such as C++ templates [75], metaprogramming has evolved into a fundamental paradigm in modern language design, as highlighted by many surveys and studies [52, 73]. Its goals are multifaceted: reducing boilerplate, enforcing structural invariants, encoding repetitive patterns and enabling compile-time specialization or reasoning that would otherwise be impossible or inefficient at run-time. Across programming ecosystems, metaprogramming serves both practical and expressive purposes, ranging from code generation and DSL construction to advanced type-level computation [73, 18]. Rust approaches metaprogramming through a powerful and expressive macro system designed to maintain the language's guarantees of safety, determinism and predictable compilation. Rust distinguishes between two primary categories: *declarative macros* and *procedural macros*.

Declarative Macros Declarative macros (defined using `macro_rules!`) operate using pattern matching over token streams, allowing developers to define custom syntactic constructs that expand into valid Rust code. While reminiscent of Lisp macros in spirit, Rust's declarative macros enforce *hygiene* [48, 49, 22]: macro-generated identifiers cannot inadvertently capture variables or introduce unintended bindings. This prevents entire classes of errors common in textual or non-hygienic macro systems.

Procedural Macros Procedural macros provide direct access to the compiler's abstract syntax tree (AST [2, 9]) through the `proc_macro` API. These macros behave as compiler plugins that transform annotated code items at compile time. Three major forms exist: *function-like procedural macros*, *attribute macros* and *derive macros*. The latter are widely used to automatically implement traits such as `Clone`, `Debug` or `Serialize`, significantly reducing repetitive boilerplate and enabling library authors to design ergonomic declarative APIs. Attribute macros allow developers to attach custom semantics or transformations to items, facilitating the creation of embedded domain-specific languages (DSLs) and compile-time validation tools; the `#[when(...)]` attribute macro introduced in this thesis is an example of such usage. Function-like procedural macros (e.g. the `spec!` macro) enable arbitrary code generation based on input token streams, allowing for complex compile-time computations and code synthesis. Because procedural macros operate on structured syntax trees instead of raw text, they can enforce well-formedness and preserve strong static guarantees during expansion.

Integration with the Compilation Pipeline Rust's macro system integrates deeply with the compiler's type checking and optimization phases. Unlike languages where macros expand at a separate preprocessing stage (e.g. the C preprocessor [28]), Rust expands macros as part of the front-end compilation pipeline, ensuring that hygiene, scoping and type resolution remain consistent. This design enables robust static analyses to occur alongside macro expansion and avoids undesired ambiguities or semantic inconsistencies. The combination of hygienic rewriting, AST-level transformations and compile-time type awareness makes Rust's metaprogramming capabilities both expressive and safe, empowering developers to construct efficient abstractions without compromising reliability.

4
A deep dive into MonoSpec

4.1 Introduction

In Chapter 2, we presented a practical example showcasing the usage of MonoSpec. This chapter dives deeper into its design principles and implementation details, elucidating the underlying mechanisms that enable stable specialization through metaprogramming. The actual implementation is in Rust, but as stated earlier, the concepts can be generalized to other programming languages with similar capabilities. First, we begin by outlining the core design goals that guided the development of the tool.

Design Goals The primary design goals for MonoSpec were centered around achieving effective specialization in stable code while adhering to the principles of safety, usability and performance. These goals shaped the tool's architecture, macro design and code generation techniques. The key objectives included:

- **Stable Compatibility**: Ensure that the tool operates entirely within the confines of stable Rust, avoiding any reliance on nightly features or unstable APIs.
- **Type Safety**: Maintain Rust's strong type safety guarantees, ensuring that specialized implementations are correctly typed and validated at compile time.
- **Ease of Use**: Provide a straightforward and intuitive interface for developers to define specialized implementations without excessive boilerplate or complexity.
- **Performance Efficiency**: Generate specialized code that is optimized for performance, minimizing overhead and ensuring that the benefits of specialization are realized in practice.

Overview As already seen, MonoSpec leverages the macro system to enable specialization. The three primary components needed by the approach are:

- **Conditions Definition**: A macro that annotates specialized implementations, specifying the conditions under which they should be used and generates the necessary boilerplate.
- **Method Wrapping**: A second macro that wraps method calls, directing them to the appropriate specialized implementation.
- **Build Script**: A way to analyze the code at compile time and collect the necessary information to facilitate specialization.

4.2 Conditions Definition

The first step to achieve specialization through metaprogramming is to define various specialized implementations that should be applied when certain conditions are met. In MonoSpec, this is accomplished using the `#[when(predicate)]` attribute macro, which takes as argument the predicate, which is the condition that must be satisfied for the specialized implementation to be selected. Please note that if we wanted a simpler drop-in replacement for the unstable specialization feature, we could avoid passing the predicate altogether and just use normal rust syntax for implementations, but we still would need to mark the implementation with some kind of macro to generate the necessary boilerplate. However, we believe that our approach of using a predicate provides more flexibility and control over the specialization process, allowing for more fine-grained optimizations and better integration with existing codebases.

Predicates In our implementation, the macro supports these predicates:

- **Type Equality**: Check if a generic type parameter is of a specific type, e.g. `T = i32`.
- **Trait Implementation**: Check if a generic type parameter implements a specific trait or multiple traits, e.g. `T: Display` or `T: Debug + Clone`.
- **Logical And**: Check if all the inner predicates are satisfied, e.g. `all(predicate1, predicate2, ...)`.
- **Logical Or**: Check if at least one of the provided predicates is satisfied, e.g. `any(predicate1, predicate2, ...)`.
- **Logical Not**: Check if the provided predicate is not satisfied, e.g. `not(predicate)`.

These predicates can be combined to form complex conditions, allowing for flexible and powerful specialization rules.

Wildcards In addition to the specific predicates, MonoSpec supports a wildcard condition that matches any type. For instance, the condition `T = Vec<_>` matches any vector type, regardless of its element type. Another useful wildcard condition is `T = &_`, which matches any reference type. Wildcards are not allowed in trait implementation predicates, as from our point of view they would not provide any useful functionality, for example knowing that a type implements any trait without specifying which one doesn't help much in specializing code.

Nested Conditions The macro also supports nested conditions, allowing for more complex specialization rules. For example, we can have a condition that checks if a type is a vector of types that implement a specific trait, e.g. `all(T = Vec<U>, U: Display)`, where both `T` and `U` are generic type parameters. Or we could have a condition that checks if a type is a reference to a type that is either `i32` or another reference to any other type, e.g. `T = &U, any(U = i32, U = &_)`, and so on. All the predicates and wildcards defined earlier can be nested within each other to form more complex conditions and allow for fine-grained control over specialization.

Default Implementation In addition to specialized implementations, MonoSpec supports a default implementation that is used when no other conditions are met. These default implementations are picked up by default by the Build Script and don't need to be explicitly marked with any macro. Optionally, if we wanted to limit the implementations available for specialization, we could use another macro to explicitly mark the default implementations.

Normalization To ensure that the conditions are consistent and can be easily compared, MonoSpec normalizes the conditions. This normalization process involves transforming the conditions into a canonical form, which allows for easier comparison and resolution of overlapping conditions. The normal form chosen is a disjunctive normal form (DNF) [59], where each condition is represented as a conjunction of predicates. This means that each condition is a logical OR of one or more predicatesand each predicate is a logical AND of one or more atomic conditions. The normalization process also involves applying De Morgan's laws [27] to push negations down to the atomic conditions, ensuring that the conditions are expressed in a consistent manner. This representation allows for efficient checking of whether a condition is satisfied, because we can easily check each disjunctive clause separately. For example, the condition `not(all(T: Display, any(T = i32, T = f64)))` is normalized to `any(not(T: Display), all(not(T = i32), not(T = f64)))`, so we know that that specific specialized implementation should be applied either if `T` does not implement `Display` or if `T` is neither `i32` nor `f64`. Some particular cases could also be completely flattened during normalization, e.g. `all(T = Vec<U>, any(U = i32, U = f64))` would be flattened to `any(T = Vec<i32>, T = Vec<f64>)` and hence an implementation would be generated for each of the two cases, as explained in the following paragraph.

Macro expansion Whenever possible, the conditions are evaluated at compile time and the generic parameters are replaced or constrained accordingly in all occurrences. For example: if we have a type equality condition, the generic parameter is replaced with the specific type in all its occurrencies; if we have a trait implementation condition, the generic parameter is constrained to implement the specified trait(s). Given that all the conditions are normalized, the macro expansion can be greatly optimized, allowing for efficient resolution of the conditions at compile time. Each disjunct condition is expanded into its own `trait` and `impl` couple, where the new trait's name is originated by appending the conditions hash to the original trait name. This is the reason why we previously defined this approach *monomorphic specialization*. For ease of read, in this thesis we will give each specialized trait a name that represents the underlying conditions, instead of the actual names the tool would output. An example of this expansion is the "Specialized implementation for MyI8 or &u8" shown in Listings 14 and 15, which we'll see in the following paragraph.

Examples Listings 14 and 15 show an example of how to define specialized implementations using the *`#[when(predicate)]`* macro and the corresponding expansion generated by the macro. The examples generate specializations for a struct `MyType` and a trait `Foo`, which takes a generic type parameter `T` and has only one method `foo(&self, x: T)`, as seen in Listing 13.

```rust
struct MyType;

trait Foo<T> {
    fn foo(&self, x: T);
}
```

Listing 13. *Definition of the simple struct and trait used in the following listings.*

```rust
// Default implementation
impl<T> Foo<T> for MyType {
    fn foo(&self, x: T) {
        println!("Default Foo for MyType");
    }
}

// Specialized implementation for Display
#[when(T: Display)]
impl<T> Foo<T> for MyType {
    fn foo(&self, x: T) {
        println!("Foo for MyType where T implements Display");
    }
}

// Specialized implementation for &[_]
#[when(T = &[_])]
impl<T> Foo<T> for MyType {
    fn foo(&self, x: T) {
        println!("Foo for MyType where T is &[_]");
    }
}

// Specialized implementation for not Vec<_>
#[when(not(T = Vec<_>))]
impl<T> Foo<T> for MyType {
    fn foo(&self, x: T) {
        println!("Foo for MyType where T is not Vec<_>");
    }
}

// Specialized implementation for MyI8 or &u8
type MyI8 = i8;

#[when(any(T = MyI8, T = &u8))]
impl<T> Foo<T> for MyType {
    fn foo(&self, x: T) {
        println!("Foo for MyType where T is MyI8 or &u8");
    }
}
```

Listing 14. *Example of the conditions definition using the tool.*

```rust
// Default implementation
impl<T> Foo<T> for MyType {
    fn foo(&self, x: T) {
        println!("Default Foo for MyType");
    }
}

// Specialized implementation for Display
trait FooDisplay {
    fn foo(&self, x: String);
}

impl<T: Display> FooDisplay<T> for MyType {
    fn foo(&self, x: T) {
        println!("Foo for MyType where T implements Display");
    }
}

// Specialized implementation for &[_]
trait FooSlice<T> {
    fn foo(&self, x: &[T]);
}

impl<T> FooSlice<T> for MyType {
    fn foo(&self, x: &[T]) {
        println!("Foo for MyType where T is &[_]");
    }
}

// Specialized implementation for not Vec<_>
trait FooNotVec<T> {
    fn foo(&self, x: T);
}

impl<T> FooNotVec<T> for MyType {
    fn foo(&self, x: T) {
        println!("Foo for MyType where T is not Vec<_>");
    }
}

// Specialized implementation for MyI8 or &u8
type MyI8 = i8;

trait FooMyI8 {
    fn foo(&self, x: MyI8);
}

impl FooMyI8 for MyType {
    fn foo(&self, x: MyI8) {
        println!("Foo for MyType where T is MyI8 or &u8");
    }
}

trait FooRefU8 {
    fn foo(&self, x: &u8);
}

impl FooRefU8 for MyType {
    fn foo(&self, x: &u8) {
        println!("Foo for MyType where T is MyI8 or &u8");
    }
}
```

Listing 15. *Example of the expansion of the conditions definition defined in Listing 14.*

4.3 Method Wrapping

To enable the selection of the appropriate specialized implementation, we need a second macro that wraps method calls. In MonoSpec, this is accomplished using the `spec!` macro, which takes as first arguments the method call itself.

Type Annotations If the language supported reflection, if we had some way to inspect the type of the arguments at compile time or if this was implemented directly in the compiler, the method call alone would have been enough. That not being the case for Rust, we need to provide additional information to the macro. This additional pieces of information include the types of object on which the method is called and the types of the arguments passed to the method, if any.

- **Type of the Object**: The type of the object on which the method is called, e.g. if we have `x.fn(arg1, arg2)`, we need to provide the type of `x`, e.g. `MyType`.
- **Types of the Arguments**: A list of the types of the arguments passed to the method, if the method takes no arguments, it can be empty or omitted, e.g. if we have `x.fn(arg1, arg2)`, we need to provide the types of `arg1` and `arg2`, e.g. `[Arg1Type, Arg2Type]`.

With this information, the tool can solve almost all of the specialization at compile time, generating a direct call to the appropriate specialized implementation.

Annotations The *type annotations* defined above are the first two of what we call *annotations*, which are additional pieces of information provided to the macro to help it resolve the specialization. In Rust, just like we can't inspect the type of a variable at compile time, there is also no way to inspect the traits implemented by a type or to know type aliases. To overcome these limitations, MonoSpec allows the user to provide additional annotations to the method wrapping macro (once again notice that if the language supported reflection or similar features this would not be necessary, even more so if specialization was implemented directly in the compiler). These additional annotations include:

- **Implemented Traits**: A list of traits implemented by each relevant type, e.g. `i32: Display` or `i32: Debug + Clone`.
- **Type Aliases**: A list of type aliases for each relevant type, e.g. `MyInt = i32`.

These annotations can be provided as additional arguments to the `spec!` macro, allowing the tool to accurately resolve the specialization even in the presence of type aliases or trait bounds in the conditions. Considering all of this, the full syntax for the `spec!` macro is the following: `spec!{ method_call; var_type; args_types; annotations... }`, where `method_call` is the method call, `var_type` is the type of the object on which the method is called, `args_types` is a list of the types of the arguments passed to the method (if any) and `annotations` are any additional annotations provided by the user. Additionally, for languages that support class inheritance, other annotations could be added to reflect hierarchy, e.g. `T1 <: T2`.

Overlap An important aspect to consider when implementing specialization is the possibility of overlapping conditions. When multiple specialized implementations could apply to a given method call, the tool needs a way to resolve the overlap and select the most appropriate implementation. One way to handle this is to define a priority system based on the specificity of the conditions. For example, a condition that checks for type equality (e.g. `T = i32`) could be considered more specific than a condition that checks for trait implementation (e.g. `T: Display`). In MonoSpec, this is effectively how overlaps are resolved, just like in Rust's unstable specialization feature. When multiple conditions of the same priority level overlap, the tool raises a compile-time error, prompting the user to resolve the ambiguity by refining the conditions. A more general and broader mechanism is the *lattice* rule defined in Rust original specialization RFC [60], which instead of requiring a strict priority system, allows partial ordering given that there is also an additional implementation that covers precisely the area of overlap. Though this approach is more flexible, it is also more complex to implement and since this is not the main priority of this thesis, we opted for the simpler priority system. The chain of priority used in MonoSpec is as follows:

1. Type Equality
2. Trait Implementation
3. Not Type Equality
4. Not Trait Implementation

$$\text{T = \&i32} > \text{T = \&U} \geq \text{T = \&_} > \text{T: Display + Debug} > \text{T: Display} > \text{not(T = \&i32)} > \text{not(T = \&U)} \geq \text{not(T = \&_)} > \text{not(T: Display + Debug)} > \text{not(T: Display)}$$

Listing 16. *Example of priority ordering of conditions, highest to lowest. U is a generic type parameter and hence overlaps with the wildcard unless it is constrained in some other conditions.*

Examples Listings 17 and 18 show an example of how to wrap method calls using the `spec!` macro and the corresponding expansion generated by the macro. The examples use the specialized implementations defined in Listings 14 and 15.

```
fn main() {
    let x = MyType;
    spec! { x.foo(vec![1, 2, 3]); MyType; [Vec<i32>] };
    spec! { x.foo("hello".to_string()); MyType; [String]; String: Display };
    spec! { x.foo(&[1, 2, 3]); MyType; [&[i32]] };
    spec! { x.foo("hello"); MyType; [&str] };
    spec! { x.foo(1i8); MyType; [i8]; MyI8 = i8 };
    spec! { x.foo(&1u8); MyType; [&u8] };
}
```

Listing 17. *Example of method wrapping, using implementations defined in Listing 14.*

```rust
fn main() {
    let x = MyType;
    <MyType as Foo<Vec<i32>>>::foo(&x, vec![1, 2, 3]);
    <MyType as FooDisplay>::foo(&x, "hello".to_string());
    <MyType as FooSlice<i32>>::foo(&x, &[1, 2, 3]);
    <MyType as FooNotVec<&str>>::foo(&x, "hello");
    <MyType as FooMyI8>::foo(&x, 1i8);
    <MyType as FooRefU8>::foo(&x, &1u8);
}
```

Listing 18. *Example of the expansion of method wrapping from Listing 17.*

```
Default Foo for MyType
Foo for MyType where T implements Display
Foo for MyType where T is &[_]
Foo for MyType where T is not Vec<_>
Foo for MyType where T is MyI8 or &u8
Foo for MyType where T is MyI8 or &u8
```

Listing 19. *Example of the output printed by the program from Listing 17.*

4.4 Build Script

The final step is to correctly handle specialization no matter the order in which the various components are defined. Without this step, if a specialized implementation is defined after a method call site that should use it, the macro would not be able to resolve the specialization correctly. As seen in Listing 5, to solve this issue with our implementation of MonoSpec we simply need to add one line to the `build.rs` file. With that, the build script analyzes the code at compile time and collects all the necessary information to handle specialization correctly. This build script scans the code for all the `#[when(predicate)]` attributes and collects the specialized implementations, as well as the default implementations and the traits implemented by each relevant type. We've also implemented a mechanism to find the *conditions definition* macros even if they are imported with a different name, by analyzing the macro definitions in the code, e.g. `use mytool::when as my_when;`. With this information, the build script generates a metadata file that is later used by the macros to resolve the method calls correctly.

Alternatives If the language didn't support build scripts or similar mechanisms, an alternative approach could be to use some additional macros to mark the default implementations and the traits explicitly. The first is required to identify the default implementations, while the second is needed because the trait's body is used to generate the monomorphized versions of the specialized implementations. To handle the order of definitions without a build script or similar mechanism, the easiest way would be to have a two-pass compilation process, where the first pass collects all the necessary information and the second one generates the specialized code. However, this approach would be less efficient and more complex to implement. Another way would be to wrap everything in a macro that collects all the necessary information, but this would be cumbersome and would not scale well for larger codebases.

4.5 Lifetimes

One of the main issues the Rust team faced when stabilizing specialization was handling lifetimes correctly. Lifetimes are a core feature of Rust's ownership system and ensuring that specialized implementations respect lifetimes is crucial for maintaining safety guarantees and soundness of the type system. It was indeed due to these challenges that specialization got stalled for a long time before being eventually marked as unstable. When designing MonoSpec, we had to carefully consider how to handle the matter.

Soundness Concerns The main soundness concern with lifetimes and specialization arises when a specialized implementation requires, implicitly or explicitly, different lifetime constraints than those present in the default implementation. For example, if a specialized implementation returns a reference with a shorter lifetime than the default implementation, it could lead to dangling references or other unsafe behavior.

The core problem is that at code generation time (the phase where MIR is lowered to LLVM IR and then to machine code) the compiler doesn't know about lifetimes (since it has already erased all lifetime information) and so it can't use them to choose which implementation should apply. For example, the code shown in Listings 20 and 21 would compile with `#![feature(specialization)]` but lead to unsound behavior, since typecheck (another compilation phase, which is at higher level and hence still has all lifetimes information) and code-gen would disagree about which implementation should be called by `x.my_method()`. Another clear case of unsoundness of specialization with lifetimes could be if the lifetime-specialized implementation returns a different type than the default implementation, causing type-check and code-gen to disagree about the type of the returned value, leading to undefined behavior.

```
trait MyTrait {
    fn my_method(&self);
}
impl<T> MyTrait for T {
    default fn my_method(&self) {
        println!("Generic implementation");
    }
}
impl<T: 'static> MyTrait for T {
    fn my_method(&self) {
        println!("Specialized for 'static");
    }
}
fn main() {
    let x = "test"; // &str is 'static
    x.my_method();  // should call specialized impl but calls the generic one
}
```

Listing 20. *Example of a soundness issue with explicit lifetime constraint.*

```rust
trait MyOtherTrait {}
impl<T: 'static> MyOtherTrait for T {}

trait MyTrait {
    fn my_method(&self);
}
impl<T> MyTrait for T {
    default fn my_method(&self) {
        println!("Generic implementation");
    }
}
impl<T: MyOtherTrait> MyTrait for T {
    fn my_method(&self) {
        println!("Specialized for 'static");
    }
}
fn main() {
    let x = "test"; // &str implements MyOtherTrait because it is 'static
    x.my_method();  // should call specialized impl but calls the generic one
}
```

Listing 21. *Example of a soundness issue with implicit lifetime constraint.*

Workarounds This soundness issue was reported shortly after the specialization proposal [1] and stalled its progress for a long time. To mitigate the issue, the Rust team proposed a workaround that allows sound specialization for a subset of use cases [61]. This solution, called `min_specialization`, is unfortunately a breaking change from current Rust code and that's one of the reasons why it didn't get stabilized. For example, the code in Listings 20 and 21 would not compile with `#![feature(min_specialization)]`. That is because the first (the explicit one) violates the second condition of the proposal ("it is fully generic with respect to lifetimes") by adding the `T: 'static` constraint; while the second (implicit one) violates the first condition ("it relies only on implied bounds") by adding the `T: MyOtherTrait` bound.

As stated earlier, the fundamental problem is that the compiler doesn't know about lifetimes at code generation time even though they could affect which implementation should be selected. The most sensible solution to this would be in our opinion to make the compiler "smarter" by simply retaining lifetime information until code generation time (even though this would require significant changes to the compiler) or at least until monomorphization, so that lifetimes could be used to select the appropriate implementation. This ides has already been brought up in the past but it was discarded as it was considered to have a "high cost" of implementation [1]. The only other accusation against it is that it would be harder to reason about lifetimes if they could influence code selection, but we believe that this is not a strong argument against it, given that many modern programming languages have already accustomed us to using generic programming features without significant issues. Hence, in MonoSpec we decided to simulate the information that the "smarter" compiler would have about lifetimes by allowing specialization based on lifetimes in *conditions definition* and providing lifetime annotations in *method wrapping*.

Conditions Definition To allow the user to define specialized implementations based on lifetimes, we extended the `#[when(predicate)]` macro to support lifetime predicates. These predicates include:

- **Type Equality**: When defining a type equality with a reference type, we can also specify lifetime constraints, e.g. `T = &'a i32`.
- **Lifetime Bound**: Check if a generic type parameter has a specific lifetime, e.g. `T: 'b`.

Other predicates could be combined with these to form more complex conditions:

- **Lifetime Equality**: Check if a lifetime parameter is equal to a specific lifetime, e.g. `'a = 'static`.
- **Lifetime Outlives**: Check if a lifetime parameter outlives another lifetime, e.g. `'a: 'b`.

Just like for the other conditions defined earlier, also these new conditions generate explicit constraints in the expanded implementations.

Method Wrapping To provide the necessary lifetime information to the method wrapping macro, we extended the `spec!` macro to accept lifetime annotations. First of all, in *type annotations* reference types can now include lifetime information, e.g. `&'a i32`. Additionally, we added a new type of annotation to specify the lifetimes of each relevant type:

- **Lifetime Bound**: The lifetimes of each relevant type, e.g. `T: 'a`.

Furthermore, we could add a new type of annotation to specify lifetime relationships:

- **Lifetime Relationships**: Specify lifetimes equality or outlives relationships, e.g. `'a: 'b` or `'a = 'static`.

```rust
trait MyTrait {
    fn my_method(&self);
}
impl<T> MyTrait for T {
    fn my_method(&self) {
        println!("Generic implementation");
    }
}
#[when(T: 'static)] // or #[when(T = &'static _)]
impl<T> MyTrait for T {
    fn my_method(&self) {
        println!("Specialized for 'static");
    }
}
fn call_method(x: &'static i32, y: &i32) {
    spec! { x.my_method(); &'static i32 } // prints "Specialized for 'static"
    spec! { y.my_method(); &i32 }         // prints "Generic implementation"
}
```

Listing 22. *Example of specialization using lifetimes.*

Higher Ranked Polymorphism Other than first-order polymorphism (i.e. generics over types), Rust also supports higher-ranked polymorphism through the use of `for<'a>` syntax. This feature allows for functions to be generic over lifetimes, enabling more flexible and reusable code. This is particularly useful in scenarios like callbacks or functional programming constructs that operate on references without being tied to a specific lifetime. To support specialization involving higher-ranked polymorphism, we extended both macros to handle higher-ranked lifetime parameters.

An example of this can be seen in Listings 23 and 24, where we define a specialized implementation for a trait `Bar` that has a method `bar` accepting a higher-ranked reference. Here we can also see how MonoSpec handles returning values from specialized implementations' functions.

```
trait Bar<T, U, V> { fn bar(&self, t: T, u: U) -> V; }
#[when(all(
    T = &str, T: 'b,
    U = for<'a> fn(T, &'a i32) -> V
))]
impl<'b, T, U, V> Bar<T, U, V> for MyType {
    fn bar(&self, t: T, u: U) -> V {
        u(t, &7)
    }
}
fn main() {
    let x = MyType;
    let lambda = |a: &str, b: &i32| a.len() as u32 + *b as u32;
    let res: u32 = spec! { x.bar("abc", lambda); MyType;
      [&str, for<'a> fn(&str, &'a i32) -> u32] }; // returns 10
}
```

Listing 23. *Example of specialization using higher-ranked polymorphism.*

```
trait Bar<T, U, V> { fn bar(&self, t: T, u: U) -> V; }
trait BarFn<'b, V> {
    fn bar(&self, t: &'b str, u: for<'a> fn(&'b str, &'a i32) -> V) -> V;
}
impl<'b, V> BarFn<'b, V> for MyType {
    fn bar(&self, t: &'b str, u: for<'a> fn(&'b str, &'a i32) -> V) -> V {
        u(t, &7)
    }
}
fn main() {
    let x = MyType;
    let lambda = |a: &str, b: &i32| a.len() as u32 + *b as u32;
    let res: u32 = <MyType as BarFn<_>>::bar(&x, "abc", lambda); // returns 10
}
```

Listing 24. *Example of expansion of higher-ranked polymorphism specialization from Listing 23.*

4.6 Limitations

While MonoSpec provides a powerful mechanism for stable specialization, it does have some limitations that should be considered:

- **Boilerplate**: The need for additional annotations in method wrapping can introduce some boilerplate, especially in complex scenarios with many types and traits involved.
- **Compile-Time Overhead**: The use of macros and build scripts can introduce some compile-time overhead, especially in large codebases with many specialized implementations.
- **Lifetimes Complexity**: While the tool provides mechanisms to handle lifetimes, the added complexity may lead to confusion or misuse if not properly understood by the user.
- **Trait Bounds**: Since the tool generates a monomorphic version of each trait involved, the ability to involve those traits in trait bounds elsewhere in the code is lost.
- **Limited Predicate Support**: Although the tool supports a variety of predicates for specialization, the current set of supported predicates may not cover all language feature (e.g. existential types or mutable references), potentially limiting the tool's applicability in certain cases.
- **Same Project Traits**: The current implementation only supports specialization for traits defined within the same crate, because we need to analyze the trait's body to generate the monomorphized versions of the specialized implementations, limiting its usability with external libraries.
- **Warnings and Errors**: Error messages generated by the macros can be less informative than native compiler errors, making debugging more challenging.

Despite these limitations, MonoSpec represents a significant step forward in enabling stable specialization through metaprogramming, providing developers with a powerful tool to optimize their code while adhering to Rust's safety guarantees. More generally, the concepts presented in this chapter can be adapted to other programming languages with similar capabilities, broadening the applicability of the approach beyond Rust.

5 Evaluation

5.1 Introduction

To validate the utility of stable specialization, we conducted a series of experiments focusing on ecosystem analysis. These evaluations aimed to identify real-world scenarios where stable specialization could provide significant benefits, particularly in terms of code maintainability. The experiments were designed to analyze a diverse set of public Rust codebases, examining their use of traits and generic programming patterns. By applying our specialization technique to these codebases, we sought to demonstrate its practical applicability and effectiveness in real-world scenarios.

Approach To conduct the experiments, we developed a static analysis tool that scans the selected Rust codebases for instances of the identified patterns, which we'll show later. More specifically, we instructed the compiler to build a custom AST starting from the HIR for each function (including trait and impls fn items) in the codebase.

After building the AST, using an heuristic we grouped together functions that could be specializable between one another. To do so, we used a naming-based heuristic combined with a same-number-of-parameters check. For example, if we had a function named `process_int` taking an `i32` parameter and another function named `process_float` taking an `f32` parameter, we would group them together as they likely represent specialized versions of a more generic function. We also split the functions based on their origin (bare function, trait items, impl items) and if they were associated with a trait, we used another similar heuristic for the two traits.

Once we had these groups, we calculated the tree edit distance of each function in the group to one another. This was done using the ZSS algorithm [80], which computes the minimum number of operations required to transform one tree into another, allowing us to quantify the structural similarity between functions' ASTs. The ZSS algorithm has a worst-case complexity of $O(n^4)$, where n is the number of nodes in the larger tree, which makes it suitable for our analysis of function ASTs, but also computationally intensive for larger trees, which explains some of the longer analysis times we had. After calculating the edit distance, we got the similarity by using this formula

$$1 - \frac{\text{dist}}{\text{max_size}}$$

where `dist` is the tree edit distance between the two functions and `max_size` is the size of the largest AST between the two.

Finally, we filtered out groups that had an average similarity lower than a certain threshold to focus on the most promising candidates for specialization. We also made sure to add a further optimization to avoid analyzing trees that had way too different sizes, as they would mathematically never reach a good enough similarity score, using the following formula

$$\frac{\text{min_size}}{\text{max_size}} < \text{threshold}$$

where `min_size` is the size of the smallest AST between the two and `threshold` is the similarity threshold set for the analysis.

This approach allowed us to systematically identify potential specialization opportunities across a wide range of Rust codebases, providing a solid foundation for evaluating the effectiveness of stable specialization in real-world scenarios. It's important to note that our analysis was based on heuristics to identify the relevant patterns and the specialization opportunities within the selected codebases. As such, there is a possibility that some patterns were overlooked or misclassified, which could affect the comprehensiveness of our findings. It's also important to note that not all instances where specialization is possible it should be applied, e.g. if the specialized versions do not provide significant performance improvements or if they complicate the code unnecessarily, like adding more boilerplate than they would reduce.

Analysis Setup The analysis was performed on a machine with an Intel Core i7-8565U 4-Core 8-Thread Processor and 16 GB of RAM, running Ubuntu 24.04 LTS. The Rust toolchain used for the analysis was nightly-2025-11-17, with our static analysis tool integrated into the compiler as a custom pass. The analysis was executed starting from November 17, 2025 and to ensure a comprehensive evaluation, it got run with two different similarity thresholds:

- **90% Similarity Threshold**: This threshold was chosen to capture a broader range of specialization opportunities, allowing for more structural differences between functions while still identifying meaningful similarities.
- **99% Similarity Threshold**: This higher threshold was selected to focus on functions that are nearly identical in structure, ensuring that only the most closely related functions were considered for specialization.

These thresholds allowed us to evaluate the effectiveness of stable specialization across different levels of similarity, providing insights into its applicability in various scenarios.

Project Selection An important step to conduct this research was to select a set of representative Rust projects from the open-source ecosystem. For this purpose, we selected some of the most downloaded crates from `crates.io` as well as some popular projects and other less known ones, ranging from code editors to broadly used libraries, ensuring that our analysis captures a diverse set of programming patterns and practices. The analyzed projects are listed in Table 5.2, along with some relevant metadata such as their GitHub stars, `crates.io` dependents and downloads and the total number of functions and traits found while analyzing the projects.

Pattern Identification To keep the analysis focused and feasible and to ensure that we could draw meaningful conclusions from the results, we concentrated on identifying specific patterns that are likely to benefit from stable specialization. These patterns include:

- **Manual Dispatch**: In the trait we have a version of the function for each type we want to support and when calling the function, we manually dispatch to the correct version, e.g. in Listing 30.
- **Monomorphized Traits**: A trait is defined for each type we want to support and each implementation provides a specialized version of the function, requiring the caller to manually select the correct trait, e.g. in Listing 31.
- **Standalone Functions**: A standalone function is defined for each type we want to support and the caller is responsible for calling the correct version, e.g. in Listing 28.
- **Inherent Impls**: A struct has inherent implementations for each type we want to support, providing specialized versions of the function, e.g. in Listing 29.

Other than these, we've also pointed out some additional patterns that we did not include in the analysis because they were less common or harder to analyze in a systematic way, such as:

- **Type Level Dispatch**: A wrapper structure is created to hold the type and the value we want to specialize on, and then we implement the trait for each disjunct specialized version of the wrapper structure, e.g. in Listing 32. This does not support default implementations.
- **Macros**: Macros are used to generate specialized implementations based on the input type, which reduces boilerplate but does not support default implementations.
- **Pattern matching blocks**: Pattern matching is used to select the appropriate code path based on the type or value.
- **If blocks**: Conditional statements are used to select the appropriate code path based on the type or value.

The baseline implementations using unstable specialization and MonoSpec specialization can be seen in Listings 26 and 27 respectively. For each of the patterns shown in the listings, we created a minimal example that demonstrates how the pattern works and the call to the default function. To call the specialized version (in the examples always the one for `i32`), for all Listings 27 to 31, we'd need some more complex logic to check the type and call the correct specialized version, which could look similar to what's shown in Listing 25. For Listing 32, the default implementation is not supported, but the call doesn't need any type check as the correct specialized version is selected at compile time based on the type of the wrapper structure.

Each pattern represents a different approach to achieving specialization in Rust and our analysis aimed to determine how stable specialization could enhance each of these approaches. It's important to note that even though each of these patterns could be implemented using stable specialization, some patterns are more likely to benefit from

```rust
fn call<T: 'static>(my_type: &MyType, v: T) {
    match std::any::TypeId::of::<T>() {
        id if id == std::any::TypeId::of::<i32>() => {
            // SAFETY: We just checked that T is i32
            let v_i32 = unsafe { std::mem::transmute_copy::<T, i32>(&v) };
            // TODO: Call the specialized version
        }
        _ => {
            // TODO: Call the default version
        }
    }
}
```

Listing 25. *Example of the type check to select the correct specialized version.*

it than others. Firstly, for all patterns stable specialization would completely remove the method call boilerplate needed to select the correct specialized version, which as we've seen in Listing 25 can be quite verbose and error-prone. Secondly, some patterns inherently introduce more complexity and boilerplate than others, making them more suitable candidates for stable specialization, like the **Monomorphized Traits** and **Manual Dispatch** patterns, which would also benefit from the removal of a whole trait and trait fn respectively. Anyways, we believe that stable specialization could provide significant benefits across all these patterns by reducing boilerplate (as we've just discussed), improving performance and enhancing code readability and maintainability. Furthermore, many of these patterns do not support certain specialization features, like specialization based purely on trait bounds or default implementations. By leveraging stable specialization, developers can achieve more efficient and maintainable code across a variety of programming patterns.

```rust
#![feature(specialization)]
struct MyType;
trait Foo<T> {
    fn foo(&self, v: T);
}
impl<T> Foo<T> for MyType {
    default fn foo(&self, v: T) {
        println!("default");
    }
}
impl Foo<i32> for MyType {
    fn foo(&self, v: i32) {
        println!("v is i32");
    }
}
fn call<T>(my_type: &MyType, v: T) {
    my_type.foo(v);
}
```

Listing 26. *Unstable Specialization.*

```rust
struct MyType;
trait Foo<T> {
    fn foo(&self, v: T);
}
impl<T> Foo<T> for MyType {
    fn foo(&self, v: T) {
        println!("default");
    }
}
#[when(T = i32)]
impl<T> Foo<T> for MyType {
    fn foo(&self, v: T) {
        println!("v is i32");
    }
}
fn call<T>(my_type: &MyType, v: T) {
    spec! { my_type.foo(v); MyType, [_] }
}
```

Listing 27. MonoSpec *Specialization.*

```rust
struct MyType;
fn foo_default<T>(x: &MyType, v: T) {
    println!("default");
}
fn foo_i32(x: &MyType, v: i32) {
    println!("v is i32");
}
fn call<T>(my_type: &MyType, v: T) {
    foo_default(my_type, v);
}
```

Listing 28. *Standalone Functions.*

```rust
struct MyType;
impl MyType {
    fn foo_default<T>(&self, v: T) {
        println!("default");
    }
    fn foo_i32(&self, v: i32) {
        println!("v is i32");
    }
}
fn call<T>(my_type: &MyType, v: T) {
    my_type.foo_default(v);
}
```

Listing 29. *Inherent Impls.*

```rust
struct MyType;
trait Foo<T> {
    fn foo_default(&self, v: T);
    fn foo_i32(&self, v: i32);
}
impl<T> Foo<T> for MyType {
    fn foo_default(&self, v: T) {
        println!("default");
    }
    fn foo_i32(&self, v: i32) {
        println!("v is i32");
    }
}
fn call<T>(my_type: &MyType, v: T) {
    my_type.foo_default(v);
}
```

Listing 30. *Manual Dispatch.*

```rust
struct MyType;
trait FooDefault<T> {
    fn foo(&self, v: T);
}
trait FooI32 {
    fn foo(&self, v: i32);
}
impl<T> FooDefault<T> for MyType {
    fn foo(&self, v: T) {
        println!("default");
    }
}
impl FooI32 for MyType {
    fn foo(&self, v: i32) {
        println!("v is i32");
    }
}
fn call<T>(my_type: &MyType, v: T) {
    FooDefault::<T>::foo(my_type, v);
}
```

Listing 31. *Monomorphized Traits.*

```rust
struct MyType;
struct MyTypeWithV<'a, T> {
    my_type: &'a MyType,
    v: T,
}
trait Foo {
    fn foo(&self);
}
impl<'a> Foo for MyTypeWithV<'a, i32> {
    fn foo(&self) {
        println!("v is i32");
    }
}
impl<'a> Foo for MyTypeWithV<'a, u32> {
    fn foo(&self) {
        println!("v is u32");
    }
}
fn call<'a, T>(my_type: &'a MyType, v: T)
where
    MyTypeWithV<'a, T>: Foo,
{
    MyTypeWithV { my_type, v }.foo();
}
```

Listing 32. *Type Level Dispatch.*

Code Reduction To provide a clearer picture of the potential benefits of stable specialization, we estimated the code reduction that could be achieved for each of the identified patterns compared to the baseline implementation. Firstly, in Table 5.1, we evaluated the overhead of each pattern in terms of lines of code compared to the baseline of native stable specialization. The code examples used to build the table are the ones we've just seen in Listings 25 to 32.

Table 5.1 – Overhead Estimates

Pattern	Defaults	Traits	Lifetimes	Generic Calls	Overhead direct call	Overhead generic call
Native Specialization	☒	☒	☐	☒	-	-
MonoSpec Specialization	☒	☒	☒	☐	+5.9%	+58.8%
Manual Dispatch	☒	☐	☐	☐	-5.9%	+47.1%
Monomorphized Traits	☒	☐	☐	☐	+17.6%	+70.6%
Inherent Impls	☒	☐	☐	☐	-29.4%	+23.5%
Standalone Functions	☒	☐	☐	☐	-41.2%	+11.8%
Type Level Dispatch	☐	☐	☐	☒	+41.2%	+41.2%

Table 5.1. *Overhead for not using Native Specialization. For each pattern, we report the implementation overhead for both direct and generic calls, together with some metadata, such as if they support a default implementation, specialization based purely on trait bounds or lifetimes and if they only require a single generic call that the compiler can resolve on its own .*

We'll first focus on direct calls. From the estimates, we can observe that native stable specialization offers significantly less code overhead across various patterns. We can also note how MonoSpec's specialization requires almost no additional boilerplate compared to native specialization, while being the only alternative to allow specialization based purely on trait bounds and lifetimes. Even though some patterns might require less overhead, they often lack expressiveness and flexibility, other than substantial features like default implementations and specialization based purely on trait bounds.

Moving on to generic calls, it's important to understand that these previously described estimates are based on minimal examples, where the method call can be called directly without any additional logic. That's not always the case in real-world scenarios, like if the method call needs to be called in a point where the use of generic programming doesn't give the developer certainty about what should actually be called. This is a problem for all patterns that do not support a single generic call, in which cases we'd need some more complex call logic, using something similar to what we've seen in Listing 25. Overall, this gives us an average overhead of 42.2% across all patterns.

If we are interested in the code reduction rather than current overhead, we can use the inverse formula:

$$\frac{Overhead}{1 + Overhead}$$

from which we obtain that if we were to convert existing code using these patterns to native stabilized specialization, we'd achieve on average a code reduction of 29.7% across all patterns, which is a significant improvement in terms of code maintainability and readability.

5.2 Results

The analysis was run with a similarity threshold of 90% and 99% and the results for the two are reported in Tables 5.3 and 5.4 respectively. For each project and threshold, we reported its analysis time in seconds, peak RSS in MB, number of unique specializable functions, from which is also reported the percentage of the total project's functions and the same is then done for unique traits and percentages.

The findings of our experimentation revealed several interesting insights into the prevalence and effectiveness of stable specialization in real-world Rust codebases. Across the analyzed projects, we identified numerous instances where stable specialization could be applied to reduce boilerplate code and improve performance. As shown in Figure 5.1, an average of more than 20% of functions in the analyzed codebases were found to be specializable at 90% similarity threshold, with some projects exhibiting even higher proportions. At 99% similarity threshold, the average was around 10%, indicating that even with stricter similarity requirements a significant number of functions could benefit from specialization.

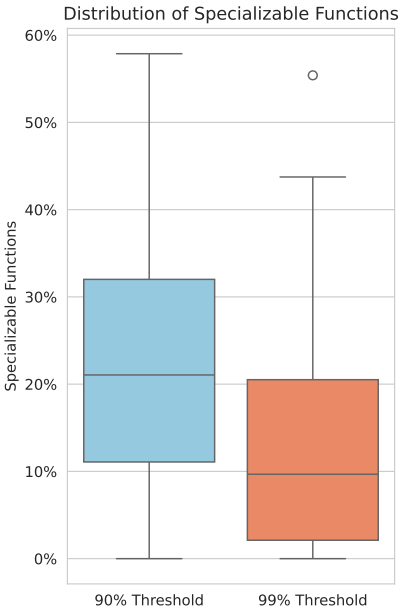


Figure 5.1. *Distribution of Spec. Fns.*

As expected based on our initial hypotheses, the major results are in projects that heavily utilize traits and generics. We also found that as the number of functions and traits in a project increases, so does the percentage of specializable instances, indicating that larger codebases stand to benefit significantly from stable specialization, as we can see in Figure 5.2.

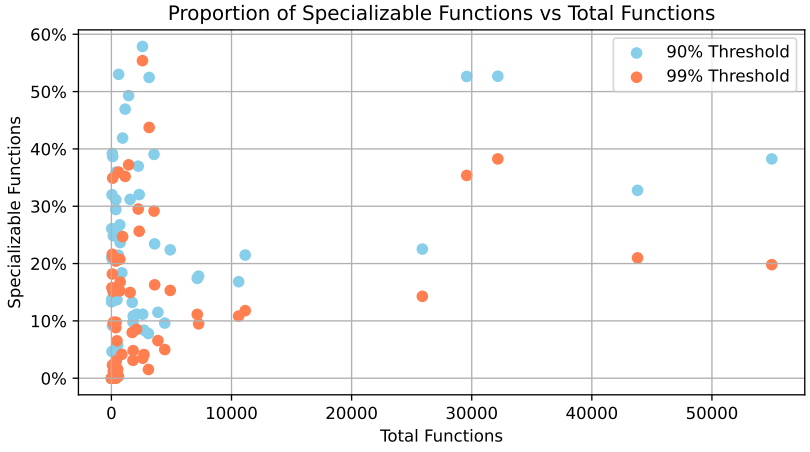


Figure 5.2. *Correlation between Number of Functions and Spec. Functions.*

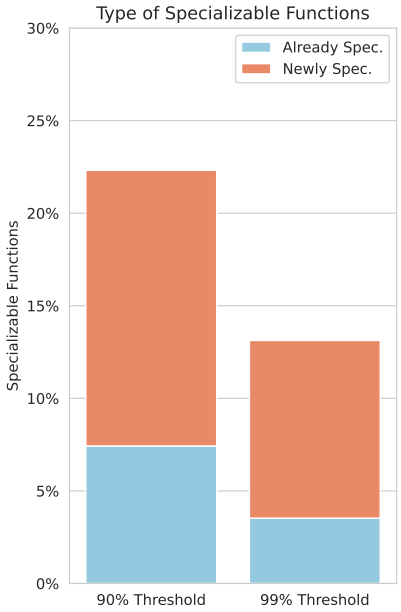


Figure 5.3. *Types of Spec. Fns.*

We then also set to understand the impact of a full stable specialization implementation versus the current subset of non-overlapping specialization that Rust offers. To do so, we identified already specializable functions as functions that satisfy our specializability heuristic defined earlier and that have a permutation of their type parameters that makes them non-overlapping. Newly specializable functions are instead functions that satisfy our heuristic but do not have any permutation of their type parameters that makes them non-overlapping and thus could only be specialized using a full stable specialization implementation. Overlap can be caused by multiple factors, such as the presence of generic type parameters, trait bounds that are not mutually exclusive or more simply equal types used in different functions. For example `fn a(x: i32, y: u32)` is already specializable with `fn b<T>(x: T, y: u32)`, because function `b`'s parameters could be permuted into a non-overlapping `fn b2<T>(y: u32, x: T)`, whatever type `T` ends up being. As shown in Figure 5.3, in both analysis we found that a full stable specialization implementation could on average triple the number of functions available for specialization compared to the current Rust specialization capabilities, highlighting the significant potential of specialization. In particular, at 90% similarity threshold, we found that on average around 67% of specializable functions were newly specializable using stable specialization, while at 99% similarity threshold, this proportion was around 73%.

Afterwards, we also tried to understand which of the patterns we identified earlier were the most common in the analyzed codebases. To do this, we first divided the functions into four distinct categories based on their structure: bare functions (not associated with any trait or impl), trait functions (defined within a trait), trait impl functions (defined within a trait impl block) and inherent impl functions (defined within an inherent impl block). As shown in Figure 5.4, we found that the majority of functions in the analyzed codebases were trait impl functions, followed by inherent impl functions and bare functions, with trait functions being by far the least common. As Figure 5.5 shows, we found that the group with the majority of specializable functions were trait impl functions, indicating that these functions were not only the most common but also the most likely to benefit from stable specialization.

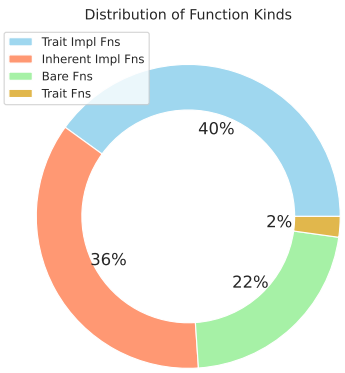


Figure 5.4. *Types of Fns.*

In particular, the most common patterns associated to trait impls are the **Monomorphized Traits** and **Manual Dispatch** patterns, hinting that many developers resort to these approaches to achieve specialization in their code. These two patterns, as seen earlier, are also the ones that would benefit the most from stable specialization, as they would both see a significant reduction in boilerplate code and complexity, even more so than other patterns. By applying stable specialization, we could achieve significant reductions in code complexity and improved maintainability.

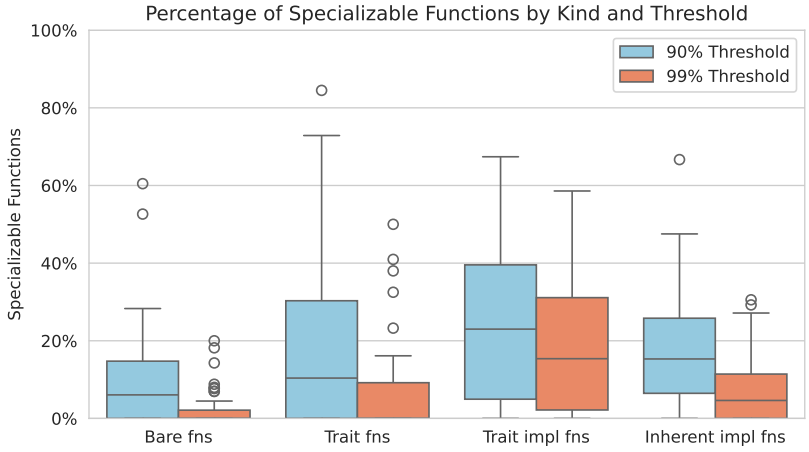


Figure 5.5. *Distribution of Spec. Functions by Kind.*

Furthermore, stable specialization could effectively handle default implementations and specialization based purely on trait bounds, which are not supported by many of the existing patterns. This capability allows developers to write more generic and reusable code without sacrificing performance.

Overall, our experimentation demonstrated that stable specialization is a valuable tool for Rust developers, offering a practical solution to common challenges associated with generic programming and code specialization.

5.3 Considerations

While our evaluation has yielded promising results regarding the potential benefits of stable specialization in Rust codebases, there are several important considerations to keep in mind when interpreting these findings.

Firstly, it's crucial to recognize that not all instances of specializable functions necessarily warrant specialization. The decision to specialize a function should be based on a careful evaluation of the trade-offs involved, including potential performance gains, code complexity and maintainability. In some cases, the overhead of specialization may outweigh the benefits, particularly if the specialized versions do not provide significant performance improvements or if they complicate the code unnecessarily. Therefore, developers should exercise discretion when applying stable specialization, ensuring that it aligns with the specific needs and goals of their projects.

Secondly, our analysis was based on heuristics to identify specializable functions and patterns within the selected codebases. As such, there is a possibility that some patterns were overlooked or misclassified, which could affect the comprehensiveness of our findings, potentially yielding false positives or negatives. Future work could involve refining these heuristics or employing more sophisticated analysis techniques to enhance the accuracy of the identification process.

One could also argue that this analysis shows the poor quality of some Rust codebases, as they resort to patterns that introduce boilerplate and complexity. While this may be true in some cases, it's important to note that as shown in Figure 5.3 the majority of the identified specializable functions could not be specialized using current Rust capabilities due to overlapping implementations. This indicates that even well-written codebases could benefit from stable specialization, as it provides a more flexible and powerful mechanism for achieving specialization without the limitations imposed by the current model. Stable specialization provides a valuable tool for developers to write more efficient and maintainable code, ultimately contributing to the overall improvement of the Rust ecosystem.

To wrap up, it's important to consider the broader implications of specialization on the ecosystem: by enabling more efficient and maintainable specialization, this feature has the potential to enhance the overall quality and performance of Rust applications across various domains.

5.4 Data

Table 5.2 – Analysis Projects

#	Name	Stars	Deps	Downloads	Fns #	Trs #
1	syn	3 215	12 774	1 169 638 606	11 140	129
2	hashbrown	2 799	1 433	948 648 240	727	34
3	bitflags	1 068	3 583	871 628 003	161	26
4	proc-macro2	880	10 815	806 768 972	460	16
5	quote	1 485	12 614	788 009 739	395	32
6	base64	697	7 686	786 716 305	84	14
7	libc	2 363	8 291	784 673 557	583	10
8	getrandom	466	1 755	748 944 236	38	3
9	rand	1 934	21 883	743 653 059	598	50
10	indexmap	2 221	3 415	705 015 407	931	42
11	cfg-if	613	2 364	702 351 853	0	0
12	serde	10 133	61 896	702 283 491	3 141	71
13	itertools	3 033	7 169	702 204 937	865	33
14	autocfg	105	69	649 484 539	50	2
15	memchr	1 261	985	640 677 127	367	9
16	itoa	349	437	635 152 777	18	3
17	json	5 373	539	616 768 830	1 432	49
18	thiserror	5 199	29 908	613 253 638	188	21
19	unicode-ident	103	83	612 334 907	15	1
20	once_cell	2 039	6 860	609 099 441	106	8
21	log	2 443	24 032	607 963 553	77	7
22	heck	569	1 175	606 082 034	23	12
23	cc	2 054	2 868	578 138 896	517	20
24	regex	3 830	14 186	576 863 005	3 597	89
25	ryu	668	217	575 741 455	43	3
26	clap	15 713	26 880	565 116 789	1 730	73
27	aho-corasick	1 155	241	564 252 056	699	26
28	smallvec	1 557	2 206	560 988 870	174	35
29	strsim	457	327	539 207 767	29	3
30	parking_lot	3 160	4 078	534 825 577	363	35
31	lazy_static	2 022	10 657	524 896 258	2	0
32	num-traits	886	5 577	516 997 081	2 585	47
33	socket2	807	606	511 596 457	350	12
34	semver	647	2 040	490 095 892	87	14
35	digest	690	924	490 088 430	1 142	164
36	either	540	1 196	489 778 929	132	19
37	version_check	53	159	489 739 708	38	2
38	rustix	1 808	326	489 109 186	1 809	39

Continued on next page

Table 5.2 – Analysis Projects

#	Name	Stars	Deps	Downloads	Fns #	Trs #
39	bytes	2 124	8 941	476 345 608	695	34
40	time	1 242	3 768	474 012 695	1 558	60
41	url	1 485	8 985	440 112 102	365	34
42	toml	930	7 698	419 494 766	2 233	122
43	futures	5 757	15 877	387 648 094	2 313	126
44	glob	564	2 294	298 438 955	32	7
45	tantivy	13 983	119	8 443 224	3 871	150
46	tauri	98 798	326	8 149 533	4 898	196
47	polars	36 050	368	6 552 820	43 805	1 476
48	cargo	14 225	125	5 437 801	4 441	108
49	bat	55 795	76	2 625 346	356	16
50	ripgrep	57 234	1	1 279 887	2 096	66
51	quiche	10 869	21	1 239 916	2 607	110
52	influxdb	30 797	25	443 557	3 547	212
53	typst	47 949	71	311 285	7 260	233
54	alacritty	61 003	0	242 002	2 710	72
55	helix	41 235	0	117 928	3 080	116
56	pueue	5 903	0	106 705	389	24
57	gitoxide	10 519	0	104 688	7 148	484
58	texture-synthesis	1 793	1	69 074	166	10
59	sendme	832	0	31 181	74	7
60	union	74 491	1	16 742	29 581	1 050
61	zed	69 551	N/A	N/A	54 980	1 674
62	ruff	43 777	N/A	N/A	25 887	640
63	hyperswitch	39 130	N/A	N/A	32 172	826
64	lapce	37 601	N/A	N/A	1 797	62
65	nushell	37 226	N/A	N/A	10 592	330

Table 5.2. *Selected Projects for Analysis. For each project, we report its name with link to the github repo, github stars, crates.io dependents and downloads. We also reported the total number of functions and traits found while analyzing the projects. The projects that are not available on crates.io (because they are not a library, but rather more complex pieces of software) have N/A in the dependents and downloads columns. The rows are sorted by downloads and then by stars .*

Table 5.3 – Analysis Results at 90% Similarity

#	Name	Time (s)	Mem (MB)	Fns #	Fns %	Trs #	Trs %
1	syn	167.12	200	2 394	21.5%	36	27.9%
2	hashbrown	0.77	106	172	23.7%	10	29.4%
3	bitflags	0.06	90	40	24.8%	16	61.5%
4	proc-macro2	3.20	96	63	13.7%	0	0.0%
5	quote	3.56	89	142	35.9%	4	12.5%
6	base64	0.04	88	0	0.0%	0	0.0%
7	libc	208.20	108	125	21.4%	0	0.0%
8	getrandom	0.13	88	8	21.1%	0	0.0%
9	rand	12.25	121	317	53.0%	15	30.0%
10	indexmap	2.43	111	390	41.9%	20	47.6%
11	cfg-if	0.00	51	0	N/A	0	N/A
12	serde	574.35	329	1 648	52.5%	27	38.0%
13	itertools	17.54	140	159	18.4%	3	9.1%
14	autocfg	0.10	82	16	32.0%	0	0.0%
15	memchr	6.15	101	108	29.4%	2	22.2%
16	itoa	0.02	76	0	0.0%	0	0.0%
17	json	10.18	119	706	49.3%	13	26.5%
18	thiserror	0.22	96	18	9.6%	1	4.8%
19	unicode-ident	0.05	83	2	13.3%	0	0.0%
20	once_cell	0.17	88	41	38.7%	5	62.5%
21	log	0.03	76	16	20.8%	3	42.9%
22	heck	0.03	75	6	26.1%	1	8.3%
23	cc	0.78	99	30	5.8%	0	0.0%
24	regex	150.15	177	843	23.4%	26	29.2%
25	ryu	0.41	78	2	4.7%	0	0.0%
26	clap	22.69	157	229	13.2%	11	15.1%
27	aho-corasick	21.76	160	174	24.9%	8	30.8%
28	smallvec	0.32	95	27	15.5%	5	14.3%
29	strsim	0.03	83	4	13.8%	0	0.0%
30	parking_lot	0.88	90	113	31.1%	8	22.9%
31	lazy_static	0.00	58	0	0.0%	0	N/A
32	num-traits	53.28	139	1 496	57.9%	10	21.3%
33	socket2	0.95	94	48	13.7%	1	8.3%
34	semver	0.38	90	8	9.2%	1	7.1%
35	digest	170.80	108	536	46.9%	43	26.2%
36	either	0.22	98	28	21.2%	3	15.8%
37	version_check	0.03	82	8	21.1%	0	0.0%

Continued on next page

Table 5.3 – Analysis Results at 90% Similarity

#	Name	Time (s)	Mem (MB)	Fns #	Fns %	Trs #	Trs %
38	rustix	6.64	329	196	10.8%	7	17.9%
39	bytes	9.64	102	186	26.8%	14	41.2%
40	time	123.95	139	486	31.2%	19	31.7%
41	url	0.88	110	8	2.2%	0	0.0%
42	toml	26.16	110	826	37.0%	27	22.1%
43	futures	14.52	158	741	32.0%	37	29.4%
44	glob	0.04	84	0	0.0%	0	0.0%
45	tantivy	521.35	253	445	11.5%	24	16.0%
46	tauri	469.72	350	1 097	22.4%	34	17.3%
47	polars	9 965.49	1 458	14 356	32.8%	388	26.3%
48	cargo	234.72	413	427	9.6%	18	16.7%
49	bat	0.24	142	6	1.7%	0	0.0%
50	ripgrep	8.37	116	234	11.2%	1	1.5%
51	quiche	32.61	172	291	11.2%	16	14.5%
52	influxdb	2 430.47	602	1 385	39.0%	56	26.4%
53	typst	197.16	376	1 293	17.8%	55	23.6%
54	alacritty	801.49	310	227	8.4%	11	15.3%
55	helix	36.18	292	240	7.8%	5	4.3%
56	pueue	655.46	1 403	58	14.9%	5	20.8%
57	gitoxide	617.86	352	1 245	17.4%	36	7.4%
58	texture-synthesis	0.19	95	2	1.2%	0	0.0%
59	sendme	2.80	211	29	39.2%	3	42.9%
60	union	13 764.80	854	15 570	52.6%	176	16.8%
61	zed	6 594.13	1 317	21 034	38.3%	256	15.3%
62	ruff	1 022.12	1 075	5 833	22.5%	127	19.8%
63	hyperswitch	19 738.40	2 635	16 947	52.7%	231	28.0%
64	lapce	114.66	380	176	9.8%	11	17.7%
65	nushell	1 155.87	2 716	1 783	16.8%	61	18.5%

Table 5.3. *Results with similarity threshold of 0.99. For each analysed project, we reported its analysis time in seconds and peak RSS in MB. The following columns are built taking the number of functions that are specializable between one another and extracting the number of unique specializable functions, of which is also reported the percentage from the total number of functions in the project. The same is then done for unique traits and relative percentage. We highlighted projects where specialization surpasses 35% or is below 5%. The rows use the same sorting from Table 5.2 .*

Table 5.4 – Analysis Results at 99% Similarity

#	Name	Time (s)	Mem (MB)	Fns #	Fns %	Trs #	Trs %
1	syn	200.97	181	1 313	11.8%	30	23.3%
2	hashbrown	1.12	106	122	16.8%	9	26.5%
3	bitflags	0.11	90	34	21.1%	15	57.7%
4	proc-macro2	1.24	94	30	6.5%	0	0.0%
5	quote	5.27	89	12	3.0%	4	12.5%
6	base64	0.06	89	0	0.0%	0	0.0%
7	libc	293.78	108	2	0.3%	0	0.0%
8	getrandom	0.04	88	6	15.8%	0	0.0%
9	rand	18.10	121	215	36.0%	11	22.0%
10	indexmap	1.96	110	230	24.7%	17	40.5%
11	cfg-if	0.00	51	0	N/A	0	N/A
12	serde	293.97	284	1 374	43.7%	20	28.2%
13	itertools	2.53	115	36	4.2%	2	6.1%
14	autocfg	0.13	83	0	0.0%	0	0.0%
15	memchr	8.64	101	75	20.4%	2	22.2%
16	itoa	0.02	76	0	0.0%	0	0.0%
17	json	11.31	119	533	37.2%	10	20.4%
18	thiserror	0.32	96	18	9.6%	1	4.8%
19	unicode-ident	0.08	83	0	0.0%	0	0.0%
20	once_cell	0.14	88	37	34.9%	5	62.5%
21	log	0.06	76	14	18.2%	3	42.9%
22	heck	0.08	75	0	0.0%	0	0.0%
23	cc	0.84	98	8	1.5%	0	0.0%
24	regex	52.22	159	586	16.3%	22	24.7%
25	ryu	0.06	76	0	0.0%	0	0.0%
26	clap	13.46	138	138	8.0%	11	15.1%
27	aho-corasick	5.22	122	145	20.7%	6	23.1%
28	smallvec	0.17	94	17	9.8%	5	14.3%
29	strsim	0.04	82	0	0.0%	0	0.0%
30	parking_lot	0.60	90	32	8.8%	4	11.4%
31	lazy_static	0.00	58	0	0.0%	0	N/A
32	num-traits	48.28	139	1 432	55.4%	5	10.6%
33	socket2	0.36	94	10	2.9%	1	8.3%
34	semver	0.11	88	2	2.3%	1	7.1%
35	digest	236.64	107	402	35.2%	34	20.7%
36	either	0.25	98	20	15.2%	2	10.5%

Continued on next page

Table 5.4 – Analysis Results at 99% Similarity

#	Name	Time (s)	Mem (MB)	Fns #	Fns %	Trs #	Trs %
37	version_check	0.08	82	0	0.0%	0	0.0%
38	rustix	6.46	329	87	4.8%	7	17.9%
39	bytes	11.11	101	106	15.3%	13	38.2%
40	time	172.47	139	233	15.0%	14	23.3%
41	url	0.35	108	0	0.0%	0	0.0%
42	toml	12.24	109	659	29.5%	20	16.4%
43	futures	7.69	158	593	25.6%	30	23.8%
44	glob	0.01	83	0	0.0%	0	0.0%
45	tantivy	11.14	251	254	6.6%	16	10.7%
46	tauri	146.85	350	750	15.3%	26	13.3%
47	polars	9 065.11	1 458	9 196	21.0%	293	19.9%
48	cargo	22.23	400	222	5.0%	14	13.0%
49	bat	0.33	142	0	0.0%	0	0.0%
50	ripgrep	3.55	111	179	8.5%	1	1.5%
51	quiche	13.10	172	91	3.5%	10	9.1%
52	influxdb	1 241.06	601	1 034	29.2%	48	22.6%
53	typst	87.11	375	689	9.5%	42	18.0%
54	alacritty	722.07	261	112	4.1%	7	9.7%
55	helix	10.73	275	47	1.5%	5	4.3%
56	pueue	0.67	176	38	9.8%	4	16.7%
57	gitoxide	79.75	202	796	11.1%	26	5.4%
58	texture-synthesis	0.09	95	2	1.2%	0	0.0%
59	sendme	0.09	191	16	21.6%	1	14.3%
60	union	5 467.20	781	10 465	35.4%	90	8.6%
61	zed	4 312.97	873	10 891	19.8%	206	12.3%
62	ruff	471.98	1 074	3 698	14.3%	97	15.2%
63	hyperswitch	12 533.30	2 642	12 309	38.3%	159	19.2%
64	lapce	71.98	367	56	3.1%	5	8.1%
65	nushell	132.82	325	1 148	10.8%	51	15.5%

Table 5.4. *Results with similarity threshold of 0.99. For each analysed project, we reported its analysis time in seconds and peak RSS in MB. The following columns are built taking the number of functions that are specializable between one another and extracting the number of unique specializable functions, of which is also reported the percentage from the total number of functions in the project. The same is then done for unique traits and relative percentage. We highlighted projects where specialization surpasses 35% or is below 5%. The rows use the same sorting from Table 5.2 .*

6 Threats to Validity

We organize our discussion following Wohlin *et al.* [79]'s taxonomy.

6.1 Construct Validity

Measurement of Key Variables Our study relies on specific metrics to evaluate the effectiveness of our approach. If these metrics do not accurately capture the constructs we intend to measure, it could threaten the validity of our conclusions.

Mitigation. We carefully selected metrics that are widely accepted in the research community and relevant to our study objectives.

Similarity Criteria The criteria used to determine similarity between code snippets may not fully capture the nuances of code functionality and intent. Hence, our similarity assessments might not reflect true equivalence in behavior.

Mitigation. We based our similarity criteria on established practices in code analysis and validated them through preliminary experiments.

6.2 Internal Validity

Assumptions about AST Structure Our approach relies on certain assumptions about the structure of Abstract Syntax Trees (ASTs) generated by the Rust compiler. If these assumptions do not hold for all codebases, it could affect the validity of our results.

Mitigation. We validated our assumptions by analyzing a diverse set of Rust projects and ensuring that our method works across different coding styles and patterns. We also documented any limitations of our approach related to AST structure.

Instrumentation and Implementation Errors Errors in the implementation of our instrumentation or analysis tools could lead to incorrect results.

Mitigation. We conducted thorough testing of our tools, including unit tests and integration tests, to identify and fix potential bugs. We also performed code reviews and sought feedback from peers to enhance the reliability of our implementation.

6.3 External Validity

Focus on open-source Rust projects Our evaluation is based on 65 open-source projects from GitHub and `crates.io`. While these projects cover a broad spectrum of real-world software, they may not capture the full variability of proprietary or industrial codebases, which could limit the generalizability of our findings.

Mitigation. Many of the analyzed projects are widely used in production and serve as dependencies for industrial systems. This increases confidence that the results generalize beyond purely academic or hobbyist software. Furthermore, since our method is language-agnostic, future work can investigate applications in other ecosystems.

Selection Bias The selection of projects may introduce bias if certain types of software or development practices are overrepresented.

Mitigation. We aimed to include a diverse set of projects by selecting repositories of varying sizes, domains and activity levels. We also documented our selection criteria transparently to allow for replication and assessment of potential biases.

6.4 Conclusion Validity

Reliability of Quantitative Results Our quantitative results depend on the accuracy of our data collection and analysis methods. Errors in data extraction, measurement or statistical analysis could lead to incorrect conclusions.

Mitigation. We employed automated tools for data collection and analysis to minimize human error. Additionally, we performed multiple runs of our experiments to ensure consistency in the results. Where applicable, we used established statistical methods to analyze the data and increase confidence in our measurements.

Robustness of Qualitative Interpretations Our qualitative interpretations of the results may be influenced by subjective biases or misinterpretations of the data.

Mitigation. To mitigate this risk, we involved multiple researchers in the interpretation process. We also cross-referenced our interpretations with existing literature to ensure alignment with established knowledge in the field.

7 Related Work

This chapter positions our contribution with respect to prior work on specialization, zero-cost abstractions and compile-time optimizations. The works surveyed here mostly introduce new compilation techniques, language mechanisms or general frameworks for specialization and metaprogramming across a wide range of languages. In contrast, this thesis develops a general metaprogramming-based approach to specialization and zero-cost abstractions, instantiated and evaluated in Rust but conceptually applicable to other languages that offer comparable compile-time mechanisms.

Specialization Techniques Beyond Rust, specialization has been explored in a variety of language and system settings. Work on extending OpenMP to support automated function specialization across translation units [31] illustrates how compiler-supported specialization can be integrated into parallel programming models. In the context of C++, template metaprogramming techniques for concept-based specialization [6] demonstrate how rich compile-time interfaces and constraints can drive the generation of specialized implementations while preserving generic code structure. At a higher level of abstraction, meta-level reuse approaches for mastering domain specialization [57] investigate reusable meta-programs and frameworks that capture domain-specific knowledge and systematically apply specialization strategies. Specialization has also been extensively studied for dynamic languages, where trace-based just-in-time compilation performs aggressive type specialization along hot execution traces to eliminate dynamic checks and improve performance [29]. Complementary work on run-time specialization for C proposes a general partial-evaluation-based framework that generates specialized versions of functions based on runtime information, enabling optimization opportunities that are not available to purely static compilation [23].

Compared to these lines of work, we do not propose new compiler infrastructures but study how far one can go using existing compile-time mechanisms and systematic metaprogramming patterns to obtain practical specializations without sacrificing maintainability.

Zero-Cost Abstractions The concept of zero-cost abstractions is a fundamental principle in systems programming, particularly in languages like C++ and Rust [36]. In C++, this principle has guided the evolution of templates and the standard library, enabling high-level generic code that compiles down to code competitive with hand-written low-level implementations [69, 4]. Rust adopts the same philosophy for its owner-

ship model, iterators and trait-based generics, emphasizing that safety-oriented and high-level abstractions should compile to code with minimal or no additional runtime overhead compared to equivalent unsafe, low-level code [36, 46, 72].

In this thesis we empirically study how concrete metaprogramming and specialization patterns affect the extent to which zero-cost abstractions are actually realized in practice.

Compile-Time Optimization Techniques While specialization is one way to achieve zero-cost abstractions, compile-time optimization techniques are strategies employed by compilers to improve the performance of programs during the compilation process. These techniques include inlining, dead code elimination and loop unrolling, among others [5, 47, 37, 9]. Modern compilers, such as LLVM, implement a variety of optimization passes to enhance the generated machine code [51]. Some research is also exploring the use of machine learning to guide optimization decisions [39].

Procedure Cloning Procedure cloning is a closely related specialization technique in which the compiler creates specialized copies of procedures for selected call sites to expose more precise information to downstream optimizations [24]. More recent work applies procedure cloning to compiled dynamic languages, demonstrating that cloning guided by runtime profiles can yield significant performance improvements by tailoring procedures to common execution contexts [38].

Our approach achieves similar effects in a source-driven way, using generic programming and metaprogramming patterns to make specialization explicit in the code rather than relying on additional cloning passes in the compiler.

Type-Level Programming Type-level programming is a paradigm that leverages the type system of a programming language to perform computations at compile time. Languages like Haskell and Scala have been pioneers in this area, providing advanced type systems that support type-level programming constructs [45, 10].

We concentrate on simple, widely available type-level constructs (such as traits, associated types or type classes) and show how they can be organized to express specialization choices that downstream compilation can often optimize away.

Metaprogramming Metaprogramming has become an essential technique in modern programming languages, allowing developers to write code that generates or manipulates other code [21]. In the Lisp family of languages, powerful macro systems have been leveraged to build complex and flexible domain-specific models, for example in bioinformatics [44]. More and more languages are incorporating metaprogramming features to enhance code reuse and reduce boilerplate [67, 12].

We build on existing, lightweight metaprogramming facilities and use them systematically to implement families of specialized implementations while keeping the abstraction surface simple for users.

Performance Benchmarking Performance benchmarking is a fundamental activity in systems programming, where latency and throughput are primary design constraints [50]. Sound benchmarking methodologies and tools are needed to obtain statistically robust measurements and avoid common pitfalls such as warm-up effects, unstable baselines and measurement noise [3].

Our evaluation follows these established guidelines and applies them consistently to compare alternative metaprogramming-based specialization strategies on realistic case studies.

Rust for Safe Systems Programming Many studies have highlighted Rust's capabilities for safe systems programming [41, 8]. Works are ongoing to build translation tools that convert C/C++ codebases to Rust, aiming to enhance safety and maintainability without sacrificing performance [74]. Rust is being particularly preferred in domains where safety and performance are critical, such as aerospace, automotive and embedded systems [64, 66].

Our experimental work is carried out in Rust and contributes to this broader effort, while the metaprogramming patterns and specialization strategies we study are formulated so that they can be adapted to other systems languages with similar compile-time capabilities.

Summary In summary, prior work provides general mechanisms for specialization, powerful type and meta-programming models and robust benchmarking methodology. Our work is complementary: we stay within the constraints of existing mainstream languages and toolchains (with Rust as our main case study), systematize metaprogramming-based specialization patterns that are directly usable by practitioners and empirically evaluate their effectiveness in delivering zero-cost abstractions in realistic scenarios.

8 Future Work

This thesis has laid the groundwork for further exploration into the optimization of specialization mechanisms in Rust. Several avenues for future research and development present themselves:

- **Predicate Support**: As discussed in Section 4.6, extending MonoSpec to support more predicates could enhance its expressiveness and utility.
- **Compiler Integration:** Further work could focus on integrating the specialization techniques directly into the Rust compiler, allowing for full stable support and wider adoption within the Rust ecosystem. Once integrated, it would be interesting to explore optimizations that could be applied to the standard library and other widely used crates.
- **Performance Benchmarking:** While this thesis has provided initial benchmarks, a more extensive suite of tests across diverse real-world applications could yield deeper insights into the performance characteristics of the proposed optimizations.
- **Overlapping Implementations:** Investigating strategies to handle overlapping implementations more effectively could further enhance the robustness and applicability of the specialization mechanism, like the *lattice* approach mentioned in section 4.3.
- **Broader Language Support:** The techniques developed in this thesis could be adapted for use in other programming languages that support macros, bringing the benefits of specialization to a wider audience.
- **User Studies:** Conducting user studies to assess the usability and developer experience when employing MonoSpec in real-world coding scenarios could provide valuable feedback for refinement.

By pursuing these directions, future work can build upon the foundation established in this thesis, advancing the state of specialization in Rust and potentially influencing other programming languages as well.

9 Conclusions

Summary of Contributions This thesis has presented a novel approach to achieving trait specialization using metaprogramming techniques that operate within languages stable features. We designed and implemented a metaprogramming technique that allows developers to define specialized implementations of trait methods for specific types or trait bounds, effectively simulating the behavior of compiler-supported specialization without incurring runtime overhead. Our evaluation demonstrated that this approach can yield significant improvements in real-world scenarios, while maintaining code clarity and safety. Additionally, we developed an analysis tool to identify common patterns in existing Rust codebases where our stable specialization mechanism could be beneficial.

Implications The proposed metaprogramming-based specialization technique has important implications for the Rust ecosystem and systems programming in general. By enabling specialization in stable Rust, we lower the barrier for developers to adopt performance optimizations that were previously inaccessible due to the instability of the native specialization feature, hopefully laying a path for its eventual stabilization. This advancement promotes the broader adoption of Rust in performance-critical applications, as developers can now leverage specialization without sacrificing stability. Furthermore, our approach highlights the potential of metaprogramming as a powerful tool for extending language capabilities, suggesting that similar techniques could be applied to other languages with metaprogramming support.

Final Remarks In conclusion, this thesis has addressed a significant gap in the Rust programming language by providing a stable and effective mechanism for trait specialization through metaprogramming. Our contributions not only enhance the performance capabilities of Rust but also demonstrate the versatility and power of metaprogramming in extending language features. We believe that this work lays a solid foundation for future advancements in specialization techniques and encourages further exploration of metaprogramming as a means to overcome language limitations.

Bibliography

[1] Aaron Turon. Shipping specialization: a story of soundness. `https://aturon.github.io/blog/2017/07/08/lifetime-dispatch/`, 2017.

[2] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Pearson/Addison-Wesley, 2006.

[3] Andrey Akinshin. *Common Benchmarking Pitfalls*. Apress, 2019.

[4] Andrei Alexandrescu. *Modern C++ Design: Generic Programming and Design Patterns Applied*. Addison-Wesley Professional, 2001.

[5] Andrew Ayers, Richard Schooler, and Robert Gottlieb. Aggressive inlining. *SIGPLAN Not.*, 32, May 1997.

[6] Bruno Bachelet, Antoine Mahul, and Loïc Yon. Template metaprogramming techniques for concept-based specialization. *Scientific Programming*, 21, 2013.

[7] Roland Backhouse, Patrik Jansson, Johan Jeuring, and Lambert Meertens. Generic programming. In S. Doaitse Swierstra, José N. Oliveira, and Pedro R. Henriques, editors, *Advanced Functional Programming*. Springer Berlin Heidelberg, 1999.

[8] Abhiram Balasubramanian, Marek S. Baranowski, Anton Burtsev, Aurojit Panda, Zvonimir Rakamarić, and Leonid Ryzhyk. System programming in rust: Beyond safety. In *Proceedings of the 16th Workshop on Hot Topics in Operating Systems*. Association for Computing Machinery, 2017.

[9] I.D. Baxter, A. Yahin, L. Moura, M. Sant'Anna, and L. Bier. Clone detection using abstract syntax trees. In *Proceedings. International Conference on Software Maintenance (Cat. No. 98CB36272)*, 1998.

[10] Olivier Blanvillain, Jonathan Immanuel Brachthäuser, Maxime Kjaer, and Martin Odersky. Type-level programming with match types. *Proc. ACM Program. Lang.*, 6, January 2022.

[11] Chandrasekhar Boyapati, Robert Lee, and Martin Rinard. Ownership Types for Safe Programming: Preventing Data Races And Deadlocks. In Satoshi Matsuoka, editor, *Proceedings of the 17th Annual ACM Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA'02)*. ACM Press, November 2002.

[12] Eugene Burmako. Scala macros: let our powers combine! on how rich syntax and static types work with metaprogramming. In *Proceedings of the 4th Workshop on Scala*. Association for Computing Machinery, 2013.

[13] Luca Cardelli and Peter Wegner. On understanding types, data abstraction, and polymorphism. In *ACM Computing Surveys*, volume 17, 1985.

[14] Giuseppe Castagna. Covariance and contravariance: Conflict without a cause. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 17, 1995.

[15] J. Cavazos and M.F.P. O'Boyle. Automatic tuning of inlining heuristics. In *SC '05: Proceedings of the 2005 ACM/IEEE Conference on Supercomputing*, 2005.

[16] Craig Chambers, David Ungar, and Elgin Lee. An efficient implementation of self, a dynamically-typed object-oriented language based on prototypes. In *OOPSLA '92*, 1992.

[17] B. F. Chang and L. K. John. Exploring the limits of inlining. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 1997.

[18] Adam Chlipala. Ur: statically-typed metaprogramming with type-level record computation. *SIGPLAN Not.*, 45, June 2010.

[19] Adam Chlipala. Certified programming with dependent types. In *Lecture Notes*, 2011. MIT Course Notes, early chapters discuss type class coherence and overlapping instance safety.

[20] David G Clarke, John M Potter, and James Noble. Ownership types for flexible alias protection. In Craig Chambers, editor, *Proceedings of 13th International Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA'98)*. ACM, October 1998.

[21] William D. Clinger and Jonathan Rees. Macros that work. In *ACM-SIGACT Symposium on Principles of Programming Languages*, 1991.

[22] William D. Clinger and Mitchell Wand. Hygienic macro technology. *Proceedings of the ACM on Programming Languages*, 4, 2020.

[23] Charles Consel and François No"el. A general approach for run-time specialization and its application to c. In *ACM-SIGACT Symposium on Principles of Programming Languages*, 1996.

[24] Keith D Cooper, Mary W Hall, and Ken Kennedy. A methodology for procedure cloning. *Computer Languages*, 19, 1993. ICCL '92.

[25] Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman, and F. Kenneth Zadeck. Efficiently computing static single assignment form and the control dependence graph. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 13, 1991.

[26] Dibyendu Das. Function inlining versus function cloning. *SIGPLAN Not.*, 38, June 2003.

[27] A. De Morgan. *Formal Logic: The Calculus of Inference, Necessary and Probable*. University Press of the Pacific, 2003.

[28] Christian J. Dietrich. Cppsig: Extracting type information for c-preprocessor macro expansions. *Proceedings of the 11th Workshop on Programming Languages and Operating Systems*, 2021.

[29] Andreas Gal, Brendan Eich, Mike Shaver, David Anderson, David Mandelin, Mohammad Reza Haghighat, Blake Kaplan, Graydon Hoare, Boris Zbarsky, Jason Orendorff, Jesse Ruderman, Edwin W. Smith, Rick Reitmaier, Michael Bebenita, Mason Chang, and Michael Franz. Trace-based just-in-time type specialization for dynamic languages. In *ACM-SIGPLAN Symposium on Programming Language Design and Implementation*, 2009.

[30] Jacques Garrigue. Programming with polymorphic variants. In *Proceedings of the International Conference on Functional Programming (ICFP)*, 1998.

[31] Giorgis Georgakoudis, Thomas R. W. Scogland, Chunhua Liao, and Bronis R. de Supinski. Extending openmp to support automated function specialization across translation units. In Michael Klemm, Bronis R. de Supinski, Jannis Klinkenberg, and Brandon Neth, editors, *OpenMP in a Modern World: From Multi-device Support to Meta Programming*. Springer International Publishing, 2022.

[32] Jean-Yves Girard. Linear Logic. *Theoretical Computer Science*, 50, 1987.

[33] Jean-Yves Girard, Yves Lafont, and Laurent Regnier. *Advances in Linear Logic*. Cambridge University Press, July 1995.

[34] Robert Griesemer, Raymond Hu, Wen Kokke, Julien Lange, Ian Lance Taylor, Bernardo Toninho, Philip Wadler, and Nobuko Yoshida. Featherweight go. *Proceedings of the ACM on Programming Languages*, 4, 2020.

[35] Cordelia Hall, Kevin Hammond, Simon L. Peyton Jones, and Philip Wadler. Type classes in haskell. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 18, 1996.

[36] Son Ho, Aymeric Fromherz, and Jonathan Protzenko. Modularity, code specialization, and zero-cost abstractions for program verification. *Proc. ACM Program. Lang.*, 7, August 2023.

[37] J.C. Huang and T. Leng. Generalized loop-unrolling: a method for program speedup. In *Proceedings 1999 IEEE Symposium on Application-Specific Systems and Software Engineering and Technology. ASSET'99 (Cat. No.PR00122)*, 1999.

[38] Robert Hus'ak, Jan Kofroň, Jakub M'isek, and Filip Zavoral. Using procedure cloning for performance optimization of compiled dynamic languages. In *International Conference on Software and Data Technologies*, 2022.

[39] Tarindu Jayatilaka, Hideto Ueno, Giorgis Georgakoudis, EunJung Park, and Johannes Doerfert. Towards compile-time-reducing compiler optimization selection via machine learning. In *50th International Conference on Parallel Processing Workshop*. Association for Computing Machinery, 2021.

[40] Simon Peyton Jones, Mark Jones, and Erik Meijer. Type classes: An exploration of the design space. In *Proceedings of the ACM SIGPLAN Haskell Workshop*, 2000.

[41] Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. Safe systems programming in rust. *Commun. ACM*, 64, March 2021.

[42] Kyo C. Kang, Sholom G. Cohen, James A. Hess, William E. Novak, and A. Spencer Peterson. Feature-Oriented Domain Analysis (FODA) Feasibility Study. Technical report, Carnegie Mellon University, November 1990.

[43] ahmet serkan karataş, halit oğuztüzün, and ali doğru. global constraints on feature models. In david cohen, editor, *proceedings of the 16th international conference on principles and practice of constraint programming (cp'10)*. springer, September 2010.

[44] Bohdan B. Khomtchouk, Edmund Weitz, and Claes Wahlestedt. The machine that builds itself: How the strengths of lisp family languages facilitate building complex and flexible bioinformatic models. *arXiv: Other Quantitative Biology*, 2016.

[45] Csongor Kiss, Tony Field, Susan Eisenbach, and Simon Peyton Jones. Higher-order type-level programming in haskell. *Proc. ACM Program. Lang.*, 3, July 2019.

[46] Steve Klabnik and Carol Nichols. The rust programming language. `https://doc.rust-lang.org/book/`, 2019.

[47] Jens Knoop, Oliver Rüthing, and Bernhard Steffen. Partial dead code elimination. In *Proceedings of the ACM SIGPLAN 1994 Conference on Programming Language Design and Implementation*. Association for Computing Machinery, 1994.

[48] Eugene Kohlbecker, Daniel P. Friedman, Matthias Felleisen, and Bruce Duba. Hygienic macro expansion. In *Proceedings of the ACM SIGPLAN Symposium on Interpreters and Interpretive Techniques*. ACM, 1992.

[49] Eugene E. Kohlbecker, Daniel P. Friedman, Matthias Felleisen, and Bruce F. Duba. Hygienic macro expansion. In *LFP '86*, 1986.

[50] Samuel Kounev, Klaus-Dieter Lange, and Joakim Von Kistowski. *Systems Benchmarking*, volume 1. Springer, 2020.

[51] Chris Lattner and Vikram Adve. Llvm: A compilation framework for lifelong program analysis and transformation. In *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*. IEEE, 2004.

[52] Yannis Lilis and Anthony Savidis. A survey of metaprogramming languages. *ACM Comput. Surv.*, 52, October 2019.

[53] Nicholas D. Matsakis and Felix S. Klock II. The rust language. In *Proceedings of the ACM SIGAda Annual Conference on High Integrity Language Technology (HILT)*, 2014.

[54] John McCarthy. Recursive functions of symbolic expressions and their computation by machine, part i. *Communications of the ACM*, 3, 1960.

[55] Robin Milner. A theory of type polymorphism in programming. *Journal of Computer and System Sciences*, 17, 1978.

[56] David R. Musser and Alexander A. Stepanov. Generic programming. In P. Gianni, editor, *Symbolic and Algebraic Computation*. Springer Berlin Heidelberg, 1989.

[57] Stefan Naujokat, Johannes Neubauer, Tiziana Margaria, and Bernhard Steffen. Meta-level reuse for mastering domain specialization. In Tiziana Margaria and Bernhard Steffen, editors, *Leveraging Applications of Formal Methods, Verification and Validation: Discussion, Dissemination, Applications*. Springer International Publishing, 2016.

[58] Martin Odersky. Observers for Linear Types. In Bernd Krieg-Brückner, editor, *Proceedings of the 4th European Symposium on Programming (ESOP'92)*. Springer, February 1992.

[59] Emil L. Post. Introduction to a general theory of elementary propositions. *American Journal of Mathematics*, 43, 1921.

[60] Rust Project Developers. Specialization rfc. `https://rust-lang.github.io/rfcs/1210-impl-specialization.html`, 2015.

[61] Rust Project Developers. Maximally minimal specialization: always applicable impls. `https://smallcultfollowing.com/babysteps/blog/2018/02/09/maximally-minimal-specialization-always-applicable-impls/`, 2018.

[62] Rust Project Developers. The orphan rules. `https://doc.rust-lang.org/reference/items/implementations.html#orphan-rules`, 2024.

[63] Rust Project Developers. Rust release channels. `https://doc.rust-lang.org/book/appendix-07-nightly-rust.html`, 2024.

[64] Lukas Seidel and Julian Beier. Bringing rust to safety-critical systems in space. *2024 Security for Space Systems (3S)*, 2024.

[65] Christoph Seidl, Tim Winkelmann, and Ina Schaefer. A Software Product Line of Feature Modeling Notations and Cross-Tree Constraint Languages. In Andreas Oberweis and Ralf H. Reussner, editors, *Proceedings of the 13th Edition of Modellierung (Modellierung'16)*. Springer, March 2016.

[66] Ayushi Sharma, Shashank Sharma, Santiago Torres-Arias, and Aravind Machiry. Rust for embedded systems: Current state, challenges and open problems (extended report), 2024.

[67] Tim Sheard and Simon L. Peyton Jones. Template meta-programming for haskell. In *ACM SIGPLAN Symposium/Workshop on Haskell*, 2002.

[68] Guy L. Steele. Making asynchronous parallelism safe for the world. In *Proceedings of the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. Association for Computing Machinery, 1989.

[69] Bjarne Stroustrup. *The Design and Evolution of C++*. Addison-Wesley, March 1994.

[70] Bjarne Stroustrup. *The C++ Programming Language*. Addison-Wesley, 2005.

[71] Peter J. Stuckey and Martin Sulzmann. A theory of overloading. *ACM Trans. Program. Lang. Syst.*, 27, November 2005.

[72] The Rust Project Developers. The rustonomicon, n.d. `https://doc.rust-lang.org/nomicon/`.

[73] Laurence Tratt. Domain specific language implementation via compile-time meta-programming. *ACM Trans. Program. Lang. Syst.*, 30, October 2008.

[74] Dhiren Tripuramallu, Swapnil Singh, Shrirang Deshmukh, Srinivas Pinisetty, Shinde Arjun Shivaji, Raja Balusamy, and Ajaganna Bandeppa. Towards a transpiler for c/c++ to safer rust. *ArXiv*, abs/2401.08264, 2024.

[75] David Vandevoorde and Nicolai M. Josuttis. *C++ Templates: The Complete Guide*. Addison-Wesley, 2002.

[76] Philip Wadler. Linear Types Can Change the World! In Manfred Broy and Cliff B. Jones, editors, *Proceedings of the 2nd Working Conference on Programming Concepts and Methods (IFIP'90)*, pages 561–582. North-Holland, April 1990.

[77] Philip Wadler and Stephen Blott. How to make ad-hoc polymorphism less ad-hoc. In *Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. ACM, 1989.

[78] Mark N. Wegman and F. Kenneth Zadeck. Constant propagation with conditional branches. In *POPL '85*. ACM, 1985.

[79] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, 2012.

[80] Kaizhong Zhang and Dennis Shasha. Simple Fast Algorithms for the Editing Distance Between Trees and Related Problems. *SIAM Journal on Computing*, 1989.